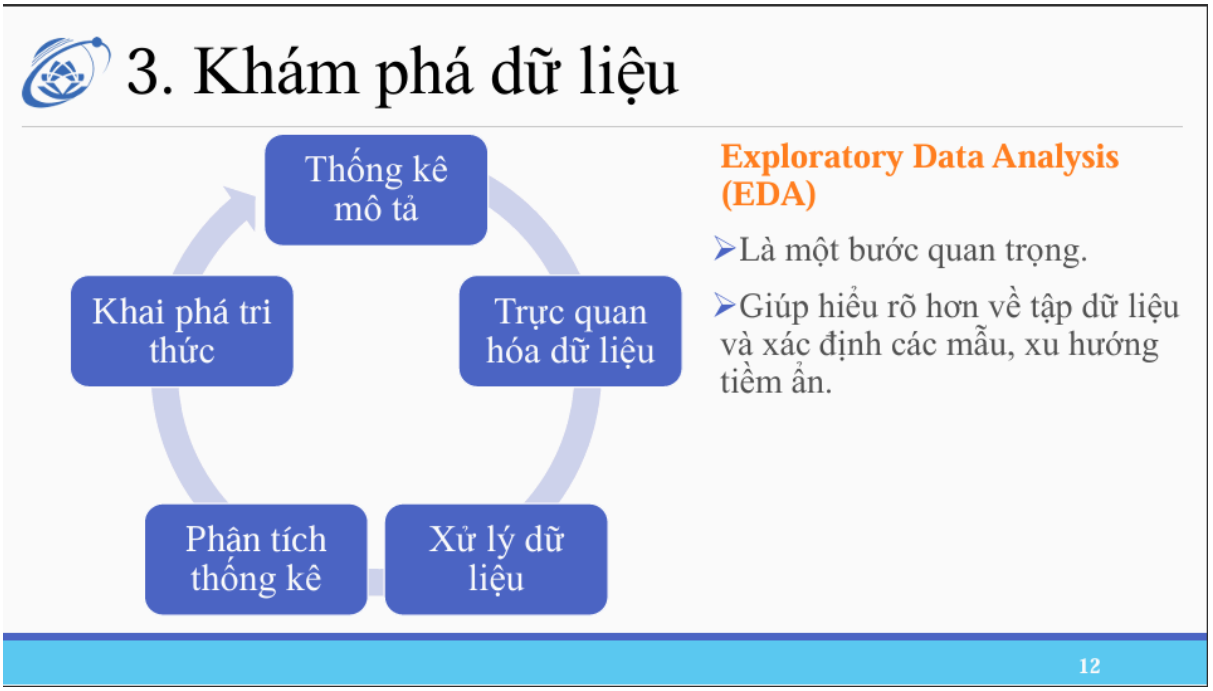
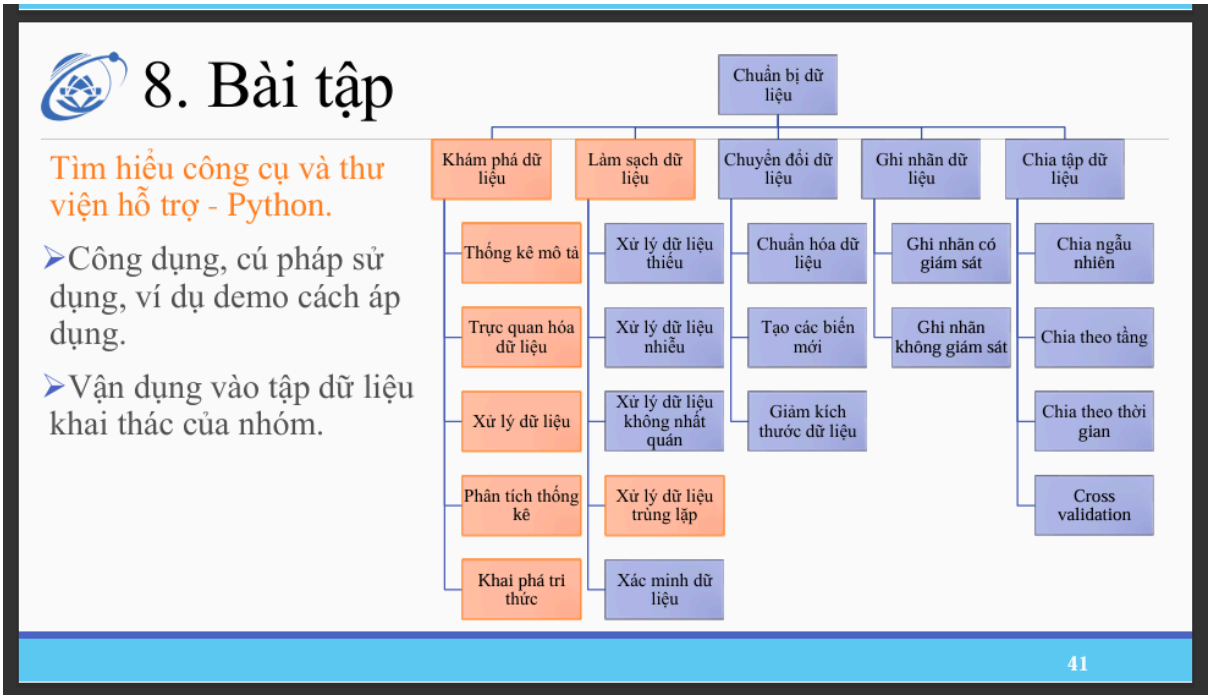
Flow Khám phá dữ liệu: **Dataset description → Data exploration → Visualization → Dimensionality reduction → Data preprocessing → Statistical analysis → Knowledge discovery → Application/insight**



Bài tập tuần (màu đỏ):



Bài tham khảo:

- [Báo cáo Bài tập chương 3 - Google Tài liệu]
- [Nhóm 4 - Bài tập chương 3 - Google Tài liệu]

Phân công:

Chi tiết công việc	Hương	Linh	Sơn	Tài	Thái
Chương 1: Giới thiệu đề tài	✓				
Chương 2: Mô tả tập dữ liệu	✓				
Chương 3: Khám phá dữ liệu		✓			
Chương 4: Thống kê mô tả dữ liệu		✓			
Chương 5: Trực quan hóa dữ liệu			✓		
Chương 6: Tiền xử lý dữ liệu			✓		
Chương 7: Xây dựng đặc trưng dữ liệu				✓	
Chương 8: Giảm chiều dữ liệu				✓	
Chương 9: Phân tích thống kê					✓
Chương 10: Khai phá tri thức					✓
Chương 11: Ứng dụng và kết luận		✓			

- **Team GG colab:** **BT3 - Code.ipynb**

Dàn bài gợi ý thư viện và công cụ phù hợp với đề tài, có thể mở rộng thêm hoặc thay đổi công cụ.

Đề tài

1. Giới thiệu đề tài

1.1 Bối cảnh nghiên cứu

Sự phát triển của các nền tảng học trực tuyến quy mô lớn (Massive Open Online Courses – MOOC) đã tạo ra một lượng dữ liệu phong phú về hành vi học tập của người học. Các dữ liệu này ghi lại chi tiết quá trình học tập, bao gồm:

- Hoạt động xem video bài giảng.
- Hoạt động làm bài tập và nộp kết quả.
- Hoạt động trao đổi qua bình luận và phản hồi.

Phân tích dữ liệu học tập (Learning Analytics) là một lĩnh vực quan trọng trong khoa học dữ liệu giáo dục. Nó cho phép:

- Hiểu rõ hơn về hành vi học tập của học sinh.
- Phát hiện các vấn đề trong quá trình học tập.
- Đề xuất giải pháp cải thiện hệ thống học trực tuyến, nâng cao chất lượng giảng dạy và hỗ trợ người học.

Trong bối cảnh đó, việc khai thác dữ liệu hành vi học tập từ MOOC để phân tích và phân loại học sinh theo mức độ tham gia (engagement) là một hướng nghiên cứu có ý nghĩa thực tiễn và học thuật.

1.2 Mục tiêu nghiên cứu

Nghiên cứu này hướng đến các mục tiêu cụ thể:

- Phân tích dữ liệu hành vi học tập của học sinh từ hệ thống MOOC.
- Khám phá các đặc trưng học tập quan trọng từ dữ liệu hành vi.
- Phân loại học sinh thành các nhóm dựa trên mức độ tham gia học tập.

Thông qua việc phân tích, nghiên cứu kỳ vọng phát hiện các nhóm học sinh có mức độ tham gia khác nhau (thấp, trung bình, cao), từ đó cung cấp cơ sở cho việc thiết kế các chiến lược hỗ trợ học tập phù hợp.

2. Mô tả tập dữ liệu

2.1 Giới thiệu tập dữ liệu

Nguồn dữ liệu sử dụng trong nghiên cứu là **MOOCCubeX**, một bộ dữ liệu lớn về học tập trực tuyến được thu thập từ nền tảng MOOC. Bộ dữ liệu này bao gồm nhiều loại thông tin:

- Hồ sơ người học.
- Hoạt động xem video bài giảng.
- Hoạt động làm bài tập.
- Các hành vi tương tác xã hội (bình luận, phản hồi).

Trong phạm vi nghiên cứu này, nhóm sử dụng năm tập dữ liệu chính:

Dataset	Nội dung ghi nhận
user.json	Hồ sơ người học: mã người dùng, giới tính, trường học, năm sinh, số lượng khóa học đã đăng ký.
user-problem.json	Hành vi làm bài tập: mã bài, số lần thử, kết quả đúng/sai, thời gian nộp.
user-video.json	Hành vi xem video: chuỗi đoạn video, thời gian xem, số lần xem.
comment.json	Hoạt động bình luận: nội dung bình luận, thời gian tạo, mã tài nguyên liên quan.
reply.json	Hoạt động phản hồi: nội dung trả lời, thời gian tạo, liên kết đến bình luận gốc.

Các tập dữ liệu này thuộc dạng **nhật ký hành vi (log data)**, tức là mỗi dòng dữ liệu ghi lại một sự kiện cụ thể của người học, ví dụ:

- Một lần xem video.
- Một lần làm bài tập.
- Một lần đăng bình luận hoặc phản hồi.

Do đó, dữ liệu ban đầu không ở dạng tổng hợp mà ở dạng sự kiện. Trước khi phân tích, cần tiến hành bước **tổng hợp dữ liệu theo từng học sinh** để xây dựng đặc trưng học tập.

2.2 Lý do lựa chọn dataset

Năm tập dữ liệu này phản ánh toàn diện các khía cạnh của quá trình học tập:

Khía cạnh	Dataset	Ý nghĩa

Hồ sơ người học	user.json	Cung cấp thông tin nền tảng để liên kết các hành vi.
Mức độ tham gia học tập	user-video.json	Cho thấy học sinh có theo dõi nội dung giảng dạy hay không.
Kết quả học tập	user-problem.json	Phản ánh khả năng hiểu bài và mức độ hoàn thành bài tập.
Tương tác xã hội – bình luận	comment.json	Thể hiện sự tham gia vào thảo luận học thuật.
Tương tác xã hội – phản hồi	reply.json	Thể hiện mức độ trao đổi, phản hồi giữa các học viên.

Kết hợp cả năm loại dữ liệu này giúp phân tích hành vi học sinh một cách toàn diện: vừa xem xét sự tham gia, vừa đánh giá kết quả học tập, đồng thời bổ sung khía cạnh tương tác xã hội. Đây là cơ sở để xây dựng mô hình phân loại nhóm học sinh theo mức độ engagement.

3. Khám phá dữ liệu

Trước khi tiến hành các bước xử lý và phân tích, nhóm nghiên cứu thực hiện khảo sát cấu trúc của tập dữ liệu để hiểu rõ đặc điểm và cách tổ chức. Bước này giúp xác định các thuộc tính quan trọng, mối quan hệ giữa các trường dữ liệu và khả năng khai thác dữ liệu trong phân tích hành vi học tập.

Trong nghiên cứu này, năm tập dữ liệu chính được sử dụng bao gồm:

- **user.json**: chứa thông tin hồ sơ người dùng trong hệ thống học trực tuyến, bao gồm mã người dùng, giới tính, trường học, năm sinh, thời gian đăng ký khóa học và số lượng khóa học đã tham gia.
- **user-problem.json**: ghi nhận quá trình người học tương tác với các bài tập, gồm dữ liệu về mã bài, số lần thử, kết quả đúng/sai và thời gian nộp. Đây là nguồn dữ liệu quan trọng để đánh giá mức độ tham gia vào hoạt động làm bài.
- **user-video.json**: lưu thông tin về hành vi xem video giảng dạy, bao gồm thời gian xem, đoạn video đã xem và các hoạt động liên quan. Tập dữ liệu này phản ánh mức độ tiếp thu kiến thức qua nội dung giảng.
- **comment.json**: chứa dữ liệu về các bình luận của người học trong hệ thống, gồm nội dung bình luận, thời gian tạo và mã tài nguyên liên quan. Đây là chỉ báo cho hoạt động thảo luận và trao đổi học thuật.
- **reply.json**: ghi nhận các phản hồi của người học đối với bình luận, gồm nội dung trả lời và thời gian tạo. Tập dữ liệu này phản ánh mức độ tương tác xã hội giữa các học viên.

Năm tập dữ liệu này kết hợp với nhau tạo thành bức tranh toàn diện về hành vi học tập: từ việc xem video, làm bài tập, đến trao đổi qua bình luận và phản hồi. Đây là cơ sở để xây dựng đặc trưng cho bài toán phân loại mức độ tham gia (engagement) của học viên.

3.2 Lý do cần khảo sát cấu trúc dữ liệu

Việc khảo sát cấu trúc dữ liệu là bước quan trọng trong quá trình phân tích dữ liệu khám phá (Exploratory Data Analysis - EDA). Nếu không hiểu rõ cấu trúc dữ liệu ban đầu, quá trình phân tích có thể gặp nhiều vấn đề như:

- lựa chọn sai các biến để phân tích
- hiểu sai ý nghĩa của các trường dữ liệu
- thực hiện các bước tiền xử lý dữ liệu không chính xác

Do đó, trong bước này cần tiến hành xem xét một số đặc điểm cơ bản của dữ liệu như:

- số lượng bản ghi trong tập dữ liệu
- số lượng thuộc tính (cột) của dữ liệu
- kiểu dữ liệu của từng thuộc tính
- mức độ đầy đủ của dữ liệu

Thông tin thu được từ bước khảo sát này sẽ giúp nhóm nghiên cứu hiểu rõ cấu trúc dữ liệu và làm cơ sở cho các bước phân tích và xử lý dữ liệu tiếp theo.

3.3 Công cụ sử dụng

Trong quá trình khảo sát và phân tích dữ liệu, nhóm nghiên cứu sử dụng các thư viện phổ biến trong Python, bao gồm:

- **Pandas**: được sử dụng để đọc, xử lý và phân tích dữ liệu dạng bảng. Thư viện này cung cấp nhiều hàm hỗ trợ kiểm tra cấu trúc dữ liệu, thống kê mô tả và xử lý dữ liệu thiếu.
- **NumPy**: được sử dụng để hỗ trợ các phép toán số học và xử lý dữ liệu số trong quá trình phân tích.

Việc sử dụng các công cụ này giúp quá trình xử lý dữ liệu trở nên thuận tiện, nhanh chóng và hiệu quả hơn khi làm việc với các tập dữ liệu có kích thước lớn.

3.4 Khám phá set User.json

a. Đọc dữ liệu

```
import pandas as pd
```

Dữ liệu trong nghiên cứu được lấy từ tập user.json thuộc bộ dữ liệu MOOC. Tập dữ liệu này lưu trữ thông tin cơ bản về người học trong hệ thống, bao gồm mã người dùng, tên, giới tính, trường học và năm sinh.

Do cấu trúc của tệp user.json có dòng đầu tiên không đúng định dạng JSON, quá trình đọc dữ liệu cần bỏ qua dòng này để tránh lỗi khi phân tích. Dữ liệu sau đó được chuyển đổi từ định dạng JSON sang cấu trúc bảng (DataFrame) bằng thư viện **Pandas** để thuận tiện cho việc phân tích.

Đoạn mã được sử dụng để đọc và xử lý dữ liệu như sau:

```python
import pandas as pd
import json

# Đọc file user.json và bỏ dòng đầu tiên
user_data = []
with open('/content/user.json', 'r') as f:
    f.readline() # bỏ dòng đầu
    for line in f:
        try:
            user_data.append(json.loads(line))
        except json.JSONDecodeError as e:
            print(f"Skipping malformed line: {line.strip()} - Error: {e}")

user_df = pd.DataFrame(user_data)
```

Trong đó:

- json.loads(line) dùng để chuyển từng dòng dữ liệu JSON thành đối tượng Python.
- Vòng lặp for line in f đọc dữ liệu từng dòng trong tệp.
- Câu lệnh try-except được sử dụng để bỏ qua các dòng dữ liệu bị lỗi.
- Cuối cùng, pd.DataFrame() chuyển dữ liệu sang dạng bảng để phục vụ quá trình phân tích dữ liệu.

Sau bước này, toàn bộ dữ liệu người dùng đã được lưu trong biến user_df.

Sau khi dữ liệu được nạp vào hệ thống, bước tiếp theo trong quá trình **Exploratory Data Analysis (EDA)** là kiểm tra cấu trúc dữ liệu. Điều này giúp hiểu rõ các thuộc tính của dữ liệu trước khi tiến hành phân tích sâu hơn.

Để quan sát dữ liệu ban đầu, hàm head() được sử dụng để hiển thị một số bản ghi đầu tiên của tập dữ liệu.

```
#xem cấu trúc dữ liệu file user.json
user_df.head()
```

	id	name	gender	school	year_of_birth	course_order	enroll_time
0	U_2244737	杨凯鸽	2	西京学院	NaN	[948109]	[2020-04-29 21:49:18]
1	U_2244738	160502011041程一	1	西京学院	NaN	[948109]	[2020-02-16 17:42:43]
2	U_2244741	160502011022王乐	2	西京学院	NaN	[948109]	[2020-04-23 11:40:57]
3	U_2244758	尚于欣	2	西京学院	NaN	[948109]	[2020-02-16 16:03:24]
4	U_2244764	蔡福政	1	西京学院	NaN	[948109]	[2020-05-03 12:55:43]

Kết quả hiển thị cho thấy mỗi bản ghi trong tập dữ liệu bao gồm các thuộc tính sau:

Thuộc tính	Ý nghĩa
id	Mã định danh của người dùng
name	Tên của người học
gender	Giới tính của người học
school	Trường học của người dùng
year_of_birth	Năm sinh của người dùng

Từ kết quả quan sát ban đầu có thể thấy:

- Mỗi dòng dữ liệu đại diện cho **một người học trong hệ thống MOOC**.
- Các trường name và school được lưu trữ dưới dạng chuỗi ký tự.
- Trường gender được biểu diễn bằng giá trị số.
- Một số trường có thể chứa giá trị thiếu, ví dụ như year_of_birth.

b. Kích thước dữ liệu

Sau khi đọc dữ liệu từ tệp user.json và chuyển sang cấu trúc DataFrame, kích thước của tập dữ liệu được kiểm tra bằng lệnh:

```
# Kiểm tra kích thước dữ liệu file user
print(user_df.shape)

(719557, 7)
```

Điều này cho thấy tập dữ liệu bao gồm **719.557 bản ghi** và **7 thuộc tính**.
Mỗi bản ghi đại diện cho một người dùng trong hệ thống học trực tuyến MOOC.

Số lượng bản ghi lớn cho thấy dữ liệu có quy mô tương đối lớn, phù hợp cho việc phân tích hành vi học tập của người dùng trong môi trường học trực tuyến.

c. Cấu trúc dữ liệu

Để hiểu rõ hơn về cấu trúc dữ liệu, hàm info() được sử dụng:

```
# Kiểm tra kiểu dữ liệu file user
user_df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 719557 entries, 0 to 719556
Data columns (total 7 columns):
 #   Column         Non-Null Count   Dtype
---  ------         --------------   -----
 0   id             719557 non-null  object
 1   name           719557 non-null  object
 2   gender         719557 non-null  int64
 3   school         719557 non-null  object
 4   year_of_birth  18475 non-null   float64
 5   course_order   719557 non-null  object
 6   enroll_time    719557 non-null  object
dtypes: float64(1), int64(1), object(5)
memory usage: 38.4+ MB
```

Kết quả cho thấy tập dữ liệu bao gồm 7 thuộc tính với các kiểu dữ liệu khác nhau:

Thuộc tính	Kiểu dữ liệu	Ý nghĩa
id	object	mã định danh của người dùng
name	object	tên người dùng
gender	int64	giới tính của người dùng
school	object	trường học của người dùng
year_of_birth	float64	năm sinh của người dùng

course_order	object	thứ tự khóa học người dùng đăng ký
enroll_time	object	thời gian đăng ký khóa học

Trong đó:

- **5 thuộc tính dạng chuỗi (object)**
- **1 thuộc tính dạng số nguyên (int64)**
- **1 thuộc tính dạng số thực (float64)**

Tổng dung lượng bộ nhớ của tập dữ liệu khoảng **38.4 MB**.

Việc xác định kiểu dữ liệu giúp hiểu cách lưu trữ thông tin và là cơ sở cho các bước xử lý dữ liệu tiếp theo.

d. Đặc điểm phân bố

Để phân tích đặc điểm phân bố của các biến số trong dữ liệu, hàm describe() được sử dụng.

```
# Thống kê mô tả file user
user_df.describe()
```

	gender	year_of_birth
count	719557.000000	18475.000000
mean	0.725749	2019.999892
std	0.829357	0.010404
min	0.000000	2019.000000
25%	0.000000	2020.000000
50%	0.000000	2020.000000
75%	1.000000	2020.000000
max	2.000000	2020.000000

Kết quả thống kê mô tả được thể hiện trong bảng sau:

Thuộc tính	Count	Mean	Std	Min	Max

gender	719557	0.73	0.83	0	2
year_of_birth	18475	2019.99	0.01	2019	2020

Từ bảng thống kê có thể rút ra một số nhận xét:

- Thuộc tính **gender** có đầy đủ dữ liệu cho tất cả người dùng.
- Giá trị của **gender** nằm trong khoảng từ **0 đến 2**, cho thấy giới tính được mã hóa dưới dạng số.
- Thuộc tính **year_of_birth** chỉ có **18.475 bản ghi hợp lệ**, thấp hơn rất nhiều so với tổng số người dùng.
- Giá trị trung bình của year_of_birth xấp xỉ **2020**, điều này cho thấy dữ liệu này có thể không phải là năm sinh thực tế mà có thể là dữ liệu bị sai lệch hoặc được ghi nhận theo một quy ước khác.

e. Dữ liệu bị thiếu

Để xác định các giá trị bị thiếu trong dữ liệu, lệnh sau được sử dụng:

```
# Kiểm tra dữ liệu thiếu
user_df.isnull().sum()
```

	0
id	0
name	0
gender	0
school	0
year_of_birth	701082
course_order	0
enroll_time	0

dtype: int64

Kết quả cho thấy:

Thuộc tính	**Số giá trị thiếu**

id	0
name	0
gender	0
school	0
year_of_birth	701082
course_order	0
enroll_time	0

Từ kết quả trên có thể nhận thấy:

- Hầu hết các thuộc tính đều **không có dữ liệu bị thiếu**.
- Tuy nhiên, thuộc tính **year_of_birth** có **701.082 giá trị bị thiếu**, chiếm phần lớn dữ liệu của cột này.

Điều này cho thấy thông tin năm sinh của người dùng không được cung cấp đầy đủ trong quá trình đăng ký hệ thống. Do tỷ lệ dữ liệu thiếu quá lớn, thuộc tính này có thể cần được:

- loại bỏ khỏi quá trình phân tích, hoặc
- xử lý bằng các phương pháp xử lý dữ liệu thiếu.

f. Nhận xét tập dữ liệu user.json

Từ quá trình khám phá dữ liệu ban đầu có thể rút ra một số nhận xét cho file user.json

- Tập dữ liệu có quy mô lớn với hơn **700 nghìn người dùng**.
- Phần lớn các thuộc tính có dữ liệu đầy đủ và có thể sử dụng trực tiếp cho phân tích.
- Thuộc tính **year_of_birth** có tỷ lệ dữ liệu thiếu rất cao nên cần được xem xét khi sử dụng trong các bước phân tích tiếp theo.
- Dữ liệu chứa thông tin về người dùng, trường học và thời gian đăng ký khóa học, đây là các thông tin quan trọng để nghiên cứu hành vi học tập trong hệ thống MOOC.

3.5 Khám phá set User-problem.json

a.Đọc dữ liệu

import pandas as pd

Tập dữ liệu user-problem.json lưu trữ thông tin về quá trình người học tương tác với các câu hỏi trong hệ thống học trực tuyến MOOC. Do kích thước của tệp dữ liệu gốc rất lớn (khoảng hàng chục GB), trong phạm vi nghiên cứu này nhóm tiến hành trích xuất **1.000.000 bản ghi đầu tiên** để phục vụ cho quá trình phân tích khám phá dữ liệu và xây dựng mô hình.

Dữ liệu được đọc từng dòng từ tệp JSON và chuyển đổi sang dạng bảng (DataFrame) bằng thư viện **Pandas**. Phương pháp đọc theo từng dòng giúp tránh lỗi tràn bộ nhớ khi xử lý các tập dữ liệu lớn.

```
# Đọc file user-problem.json
problem_records = []
valid_count = 0
max_records = 1000000

with open('/content/user-problem.json', 'r') as f:
    for line in f:
        if valid_count >= max_records:
            break
        try:
            record = json.loads(line)
            problem_records.append(record)
            valid_count += 1
        except json.JSONDecodeError:
            continue

user_problem_df = pd.DataFrame(problem_records)
print(f"DataFrame Shape: {user_problem_df.shape}")
display(user_problem_df.head())
```

```
DataFrame Shape: (1000000, 7)
```

	log_id	problem_id	user_id	is_correct	attempts	score	submit_time
0	10000_6906522	Pm_6906522	U_10000	0	1	NaN	2020-10-27 10:11:56
1	10000_6906523	Pm_6906523	U_10000	0	1	NaN	2020-10-27 10:12:13
2	10000_6906524	Pm_6906524	U_10000	1	1	NaN	2020-10-27 10:12:28
3	10000_6906525	Pm_6906525	U_10000	0	1	NaN	2020-10-27 10:14:56
4	10000_6906526	Pm_6906526	U_10000	0	1	NaN	2020-10-27 10:15:18

Kiểm tra thử 5 dòng đầu của dữ liệu:

```
#Kiểm tra 5 dòng đầu của file user-problem
user_problem_df.head()
```

	log_id	problem_id	user_id	is_correct	attempts	score	submit_time
0	10000_6906522	Pm_6906522	U_10000	0	1	NaN	2020-10-27 10:11:56
1	10000_6906523	Pm_6906523	U_10000	0	1	NaN	2020-10-27 10:12:13
2	10000_6906524	Pm_6906524	U_10000	1	1	NaN	2020-10-27 10:12:28
3	10000_6906525	Pm_6906525	U_10000	0	1	NaN	2020-10-27 10:14:56
4	10000_6906526	Pm_6906526	U_10000	0	1	NaN	2020-10-27 10:15:18

b. Kích thước dữ liệu

Sau khi đọc dữ liệu, tập dữ liệu thu được có kích thước:

```
# Kiểm tra kích thước dữ liệu file problem

print(user_problem_df.shape)

(1000000, 7)
```

Điều này cho thấy tập dữ liệu bao gồm **1.000.000 bản ghi** và **7 thuộc tính**.

c. Cấu trúc dữ liệu

Để kiểm tra cấu trúc dữ liệu, user_problem_df.info()được sử dụng:

```
#Kiểm tra kiểu dữ liệu file problem
user_problem_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000000 entries, 0 to 999999
Data columns (total 7 columns):
 #   Column       Non-Null Count    Dtype
---  ------       --------------    -----
 0   log_id       1000000 non-null  object
 1   problem_id   1000000 non-null  object
 2   user_id      1000000 non-null  object
 3   is_correct   1000000 non-null  int64
 4   attempts     1000000 non-null  int64
 5   score        520534 non-null   float64
 6   submit_time  1000000 non-null  object
dtypes: float64(1), int64(2), object(4)
memory usage: 53.4+ MB
```

Kết quả kiểm tra cấu trúc dữ liệu cho thấy tập dữ liệu bao gồm các thuộc tính sau:

Thuộc tính	Kiểu dữ liệu	Ý nghĩa
log_id	object	mã định danh của bản ghi tương tác
problem_id	object	mã câu hỏi mà người học thực hiện
user_id	object	mã người học
is_correct	int64	kết quả trả lời (1: đúng, 0: sai)
attempts	int64	số lần thử làm câu hỏi
score	float64	điểm số đạt được
submit_time	object	thời điểm người học nộp bài

Trong tập dữ liệu:

- **4 thuộc tính dạng chuỗi (object)**
- **2 thuộc tính dạng số nguyên (int64)**
- **1 thuộc tính dạng số thực (float64)**

Tổng dung lượng bộ nhớ của tập dữ liệu khoảng **53.4 MB**.

Các thuộc tính này cung cấp thông tin quan trọng về **hành vi làm bài của người học**, bao gồm kết quả trả lời, số lần thử và thời gian nộp bài.

d. Đặc điểm phân bố

Thống kê mô tả được thực hiện đối với các thuộc tính dạng số nhằm đánh giá đặc điểm phân bố của dữ liệu.

```
#Thống kê mô tả file problem
user_problem_df.describe()
```

	is_correct	attempts	score
count	1000000.000000	1000000.000000	520534.000000
mean	0.819730	1.058975	1.511911
std	0.384412	0.700367	4.402729
min	0.000000	1.000000	-1.000000
25%	1.000000	1.000000	1.000000
50%	1.000000	1.000000	1.000000
75%	1.000000	1.000000	1.000000
max	1.000000	121.000000	50.000000

Thuộc tính	Count	Mean	Std	Min	Max
is_correct	1,000,000	0.8197	0.3844	0	1
attempts	1,000,000	1.0589	0.7004	1	121
score	520,534	1.5119	4.4027	-1	50

Từ kết quả thống kê có thể rút ra một số nhận xét:

- Giá trị trung bình của **is_correct ≈ 0.82**, cho thấy phần lớn câu hỏi được người học trả lời đúng.
- Thuộc tính **attempts** có giá trị trung bình khoảng **1.06**, điều này cho thấy đa số người học chỉ thử làm câu hỏi một lần.
- Giá trị lớn nhất của **attempts = 121**, cho thấy một số câu hỏi có thể được thử nhiều lần.

- Thuộc tính **score** có giá trị trung bình khoảng **1.51**, tuy nhiên giá trị tối thiểu là **-1**, điều này có thể phản ánh các trường hợp dữ liệu đặc biệt hoặc chưa được chấm điểm.

e. Dữ liệu bị thiếu

Để kiểm tra mức độ đầy đủ của dữ liệu, nhóm tiến hành xác định số lượng giá trị bị thiếu trong từng thuộc tính bằng hàm isnull() của thư viện Pandas.Kết quả kiểm tra được thể hiện trong bảng sau:

```
#Kiểm tra dữ liệu thiếu file problem
user_problem_df.isnull().sum()
```

	0
log_id	0
problem_id	0
user_id	0
is_correct	0
attempts	0
score	479466
submit_time	0

dtype: int64

Kết quả cho thấy hầu hết các thuộc tính trong tập dữ liệu đều **không có giá trị bị thiếu**. Tuy nhiên, thuộc tính **score** có **479.466 giá trị bị thiếu** trong tổng số **1.000.000 bản ghi**.

Điều này cho thấy chỉ khoảng **52% bản ghi có thông tin về điểm số**, trong khi phần còn lại không có dữ liệu về điểm. Nguyên nhân có thể là do một số bài tập trong hệ thống chỉ ghi nhận **kết quả đúng/sai** mà không lưu trữ điểm số cụ thể, hoặc hệ thống chưa thực hiện việc chấm điểm cho các câu hỏi này.

Do đó, trong quá trình phân tích tiếp theo, thuộc tính **score** cần được xem xét xử lý phù hợp, chẳng hạn như loại bỏ các giá trị thiếu hoặc chỉ sử dụng các bản ghi có điểm số hợp lệ khi cần thiết.

f. Nhận xét tập dữ liệu user-problem.json

Từ quá trình khảo sát và phân tích khám phá dữ liệu ban đầu, có thể rút ra một số nhận xét quan trọng về tập dữ liệu user-problem:

- Tập dữ liệu có quy mô lớn với **1.000.000 bản ghi**, phản ánh quá trình tương tác của người học với các câu hỏi trong hệ thống MOOC.
- Các thuộc tính trong dữ liệu cung cấp thông tin quan trọng về **hành vi làm bài của người học**, bao gồm kết quả trả lời, số lần thử và thời gian nộp bài.
- Hầu hết các thuộc tính đều đầy đủ và không có dữ liệu thiếu, ngoại trừ thuộc tính **score**, trong đó khoảng **48% dữ liệu bị thiếu**.
- Nhìn chung, tập dữ liệu có chất lượng tương đối tốt và có thể sử dụng hiệu quả cho các bước phân tích tiếp theo.

Dữ liệu này đóng vai trò quan trọng trong việc **phân tích hành vi học tập của người dùng và xây dựng các đặc trưng phục vụ cho bài toán phân loại học sinh** dựa trên mức độ tham gia và kết quả học tập trong hệ thống học trực tuyến.

3.6 Khám phá tập dữ liệu user-video.json

a. Đọc dữ liệu

Dữ liệu trong nghiên cứu được lấy từ tập **user-video.json** thuộc bộ dữ liệu MOOC. Tập dữ liệu này lưu trữ thông tin về **hành vi xem video của người học**, bao gồm mã người dùng và chuỗi các video mà người dùng đã xem trong quá trình học tập.

Trong tệp dữ liệu này, mỗi bản ghi tương ứng với một người dùng và chứa hai trường chính:

- user_id: mã định danh của người học
- seq: danh sách các video mà người học đã xem, bao gồm thông tin video_id và các đoạn thời gian xem (segment).

Để đọc dữ liệu, tệp JSON được xử lý theo từng dòng và chuyển đổi sang cấu trúc bảng **DataFrame** bằng thư viện **Pandas**. Ngoài ra, để tránh quá tải bộ nhớ khi xử lý dữ liệu lớn, nghiên cứu chỉ đọc tối đa **1.000.000 bản ghi** từ tệp dữ liệu.

Đoạn mã được sử dụng để đọc dữ liệu như sau:

```python
import json
import pandas as pd
#Đọc file user-video
video_records = []
valid_count = 0
max_records = 1000000

with open('/content/user-video.json', 'r') as f:
    for line in f:
        if valid_count >= max_records:
            break

        try:
            record = json.loads(line)
            video_records.append(record)
            valid_count += 1

        except json.JSONDecodeError:
            continue

user_video_df = pd.DataFrame(video_records)

print(user_video_df.shape)
user_video_df.head()
```

```
(25572, 2)
```

	seq	user_id
0	[{'video_id': 'V_1395633', 'segment': [{'start...	U_112
1	[{'video_id': 'V_6224262', 'segment': [{'start...	U_150
2	[{'video_id': 'V_6432959', 'segment': [{'start...	U_172
3	[{'video_id': 'V_6334508', 'segment': [{'start...	U_189
4	[{'video_id': 'V_1358540', 'segment': [{'start...	U_197

Trong đó:

- json.loads(line) dùng để chuyển từng dòng JSON thành đối tượng Python.
- Vòng lặp for line in f đọc dữ liệu từng dòng trong tệp.
- Biến max_records được sử dụng để giới hạn số lượng bản ghi được đọc.
- Câu lệnh try-except giúp bỏ qua các dòng dữ liệu bị lỗi khi đọc JSON.
- Cuối cùng, pd.DataFrame(video_records) chuyển dữ liệu sang dạng bảng để phục vụ quá trình phân tích.

Sau bước này, toàn bộ dữ liệu được lưu trong biến **user_video_df**.

Để quan sát dữ liệu ban đầu, hàm head() được sử dụng nhằm hiển thị một số bản ghi đầu tiên của tập dữ liệu.

Kết quả cho thấy mỗi bản ghi bao gồm hai thuộc tính chính:

Thuộc tính	Ý nghĩa
seq	danh sách các video mà người học đã xem
user_id	mã định danh của người học

Từ kết quả quan sát ban đầu có thể thấy:

- Mỗi dòng dữ liệu đại diện cho **một người dùng trong hệ thống MOOC**.
- Thuộc tính seq chứa **danh sách các video và các đoạn thời gian người học xem video**.
- Mỗi video có thể chứa nhiều đoạn xem khác nhau (segment), thể hiện người học có thể xem lại hoặc tua video nhiều lần.

b. Kích thước dữ liệu

Sau khi đọc dữ liệu từ tệp **user-video.json** và chuyển sang cấu trúc DataFrame, kích thước của tập dữ liệu được kiểm tra bằng lệnh user_video_df.shape

```
# Kích thước dữ liệu user-video
print(user_video_df.shape)

(25572, 2)
```

Kết quả cho thấy tập dữ liệu bao gồm:

- **25.572 bản ghi**
- **2 thuộc tính**

Mỗi bản ghi đại diện cho một người học cùng với toàn bộ lịch sử xem video của người đó.

Mặc dù số lượng thuộc tính không nhiều, nhưng trường seq chứa dữ liệu dạng **danh sách lồng nhau (nested structure)** nên thông tin thực tế của dữ liệu khá lớn và phức tạp.

c. Cấu trúc dữ liệu

Để hiểu rõ hơn về cấu trúc dữ liệu, hàm info() được sử dụng user_video_df.info()

```
#Cấu trúc dữ liệu file user-video
user_video_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 25572 entries, 0 to 25571
Data columns (total 2 columns):
 #   Column   Non-Null Count  Dtype
---  ------   --------------  -----
 0   seq      25572 non-null  object
 1   user_id  25572 non-null  object
dtypes: object(2)
memory usage: 399.7+ KB
```

Kết quả cho thấy tập dữ liệu bao gồm 2 thuộc tính với kiểu dữ liệu như sau:

Thuộc tính	Kiểu dữ liệu	Ý nghĩa
seq	object	danh sách các video và đoạn xem video
user_id	object	mã định danh người dùng

Trong đó:

- Cả hai thuộc tính đều có kiểu dữ liệu **object**.
- Không có giá trị bị thiếu trong hai cột dữ liệu.
- Dung lượng bộ nhớ của tập dữ liệu khoảng **399.7 KB**.

Tuy nhiên, trường seq có cấu trúc phức tạp vì chứa danh sách các đối tượng JSON bên trong. Do đó, cần thực hiện bước **trích xuất dữ liệu chi tiết về hành vi xem video** để phục vụ phân tích.

d. Đặc điểm phân bố

Kết quả thống kê mô tả:

```
#Thống kê mô tả file problem
user_problem_df.describe()
```

	is_correct	attempts	score
count	1000000.000000	1000000.000000	520534.000000
mean	0.819730	1.058975	1.511911
std	0.384412	0.700367	4.402729
min	0.000000	1.000000	-1.000000
25%	1.000000	1.000000	1.000000
50%	1.000000	1.000000	1.000000
75%	1.000000	1.000000	1.000000
max	1.000000	121.000000	50.000000

- **is_correct**
  - Trung bình: 0.82 → phần lớn học sinh làm đúng.
  - Độ lệch chuẩn: 0.38 → dữ liệu phân hóa rõ rệt giữa nhóm làm đúng và nhóm làm sai.
  - Min = 0, Max = 1 → đúng như kỳ vọng (biến nhị phân).
- **attempts**
  - Trung bình: 1.06 → đa số học sinh chỉ thử 1 lần.
  - Min = 1, Max = 121 → có một số trường hợp đặc biệt thử rất nhiều lần.
  - Phân phối lệch phải, với nhiều outliers ở giá trị cao.
- **score**
  - Trung bình: 1.51, độ lệch chuẩn lớn (4.40).
  - Min = -1, Max = 50 → có giá trị bất thường (score âm).
  - Phần lớn học sinh đạt điểm 1, nhưng có một số ít đạt điểm rất cao.

e. Dữ liệu bị thiếu

Kết quả kiểm tra bằng isnull().sum() cho thấy:

```
#Kiểm tra dữ liệu thiếu file problem
user_problem_df.isnull().sum()
```

	0
log_id	0
problem_id	0
user_id	0
is_correct	0
attempts	0
score	479466
submit_time	0

dtype: int64

- user_video_df không có giá trị thiếu ở cả hai cột seq và user_id.
- video_stats cũng không có dữ liệu thiếu trong cột video_watch_count.

Điều này chứng tỏ dữ liệu khá sạch, có thể sử dụng trực tiếp cho phân tích.

f. Nhận xét tập dữ liệu

- Tập dữ liệu có quy mô rất lớn (10 triệu bản ghi), phản ánh chi tiết hành vi làm bài của người học.
- Biến is_correct cho thấy phần lớn học sinh làm đúng, nhưng vẫn có nhóm làm sai nhiều.
- Biến attempts cho thấy đa số chỉ thử một lần, nhưng có một số học sinh thử rất nhiều lần (outliers).
- Biến score có phân phối bất thường, tồn tại giá trị âm và một số điểm rất cao, cần được xử lý hoặc kiểm tra lại.
- Dữ liệu sạch, không có giá trị thiếu, thuận lợi cho các bước phân tích tiếp theo.

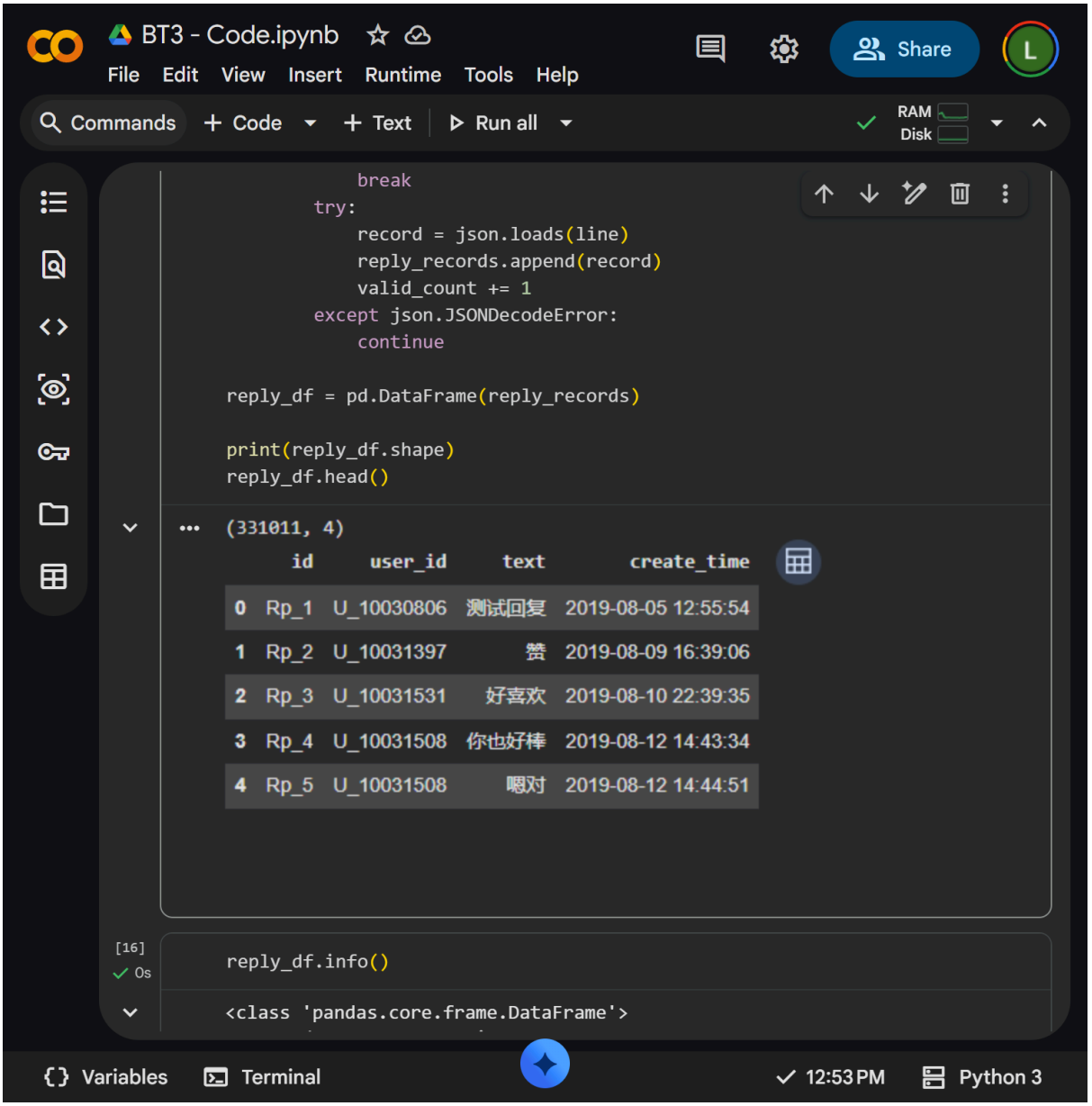
Bạn muốn viết phần **Chương 3.4 – Khám phá tập dữ liệu reply.json** theo cùng cấu trúc như ví dụ ở *user.json*. Dưới đây là bản mẫu báo cáo dựa trên kết quả bạn đã chạy:

3.7 Khám phá tập dữ liệu reply.json

a. Đọc dữ liệu

import pandas as pd

import json



Dữ liệu trong nghiên cứu được lấy từ tập **reply.json** thuộc bộ dữ liệu MOOC. Tập dữ liệu này lưu trữ thông tin về các phản hồi (reply) của người học trong hệ thống, bao gồm mã phản hồi, mã người dùng, nội dung phản hồi và thời gian tạo.

Do kích thước dữ liệu lớn, quá trình đọc được thực hiện theo từng dòng JSON, kết hợp với câu lệnh `try-except` để bỏ qua các dòng lỗi. Sau khi đọc, dữ liệu được chuyển đổi sang cấu trúc bảng (DataFrame) bằng thư viện Pandas để thuận tiện cho việc phân tích.

Kết quả sau khi đọc dữ liệu cho thấy mỗi bản ghi bao gồm các thuộc tính sau:

Thuộc tính	Ý nghĩa

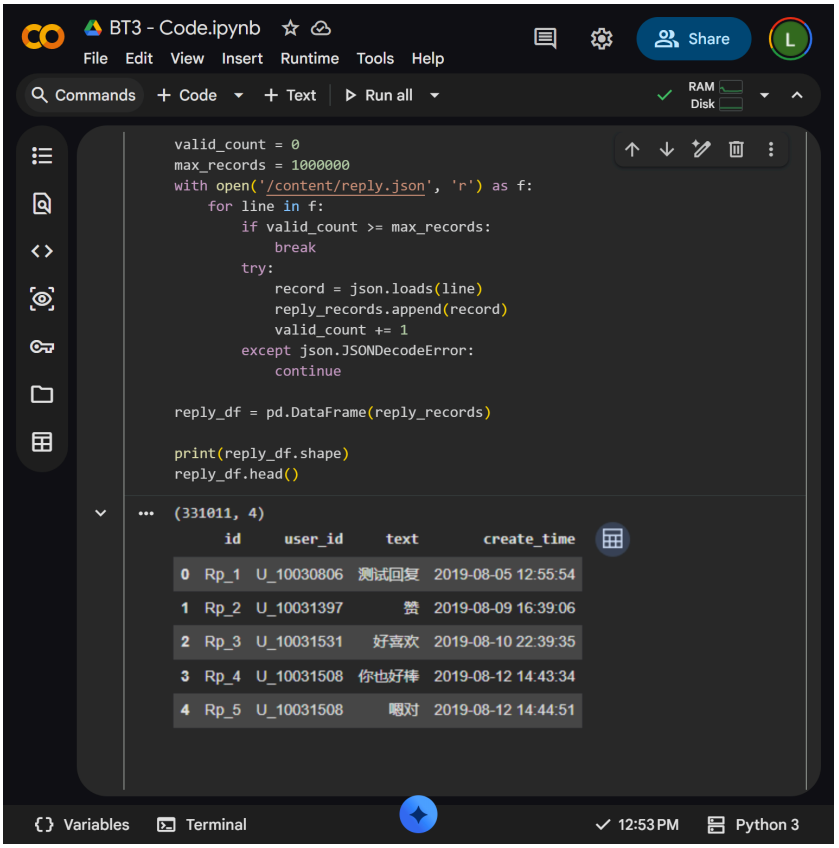
id	Mã định danh của phản hồi
user_id	Mã định danh của người dùng
text	Nội dung phản hồi
create_time	Thời gian tạo phản hồi

Quan sát dữ liệu ban đầu bằng `head()` cho thấy mỗi dòng dữ liệu đại diện cho một phản hồi của người học trong hệ thống MOOC.

b. Kích thước dữ liệu

Sau khi đọc dữ liệu từ tệp **reply.json** và chuyển sang DataFrame, kích thước của tập dữ liệu được kiểm tra bằng lệnh:

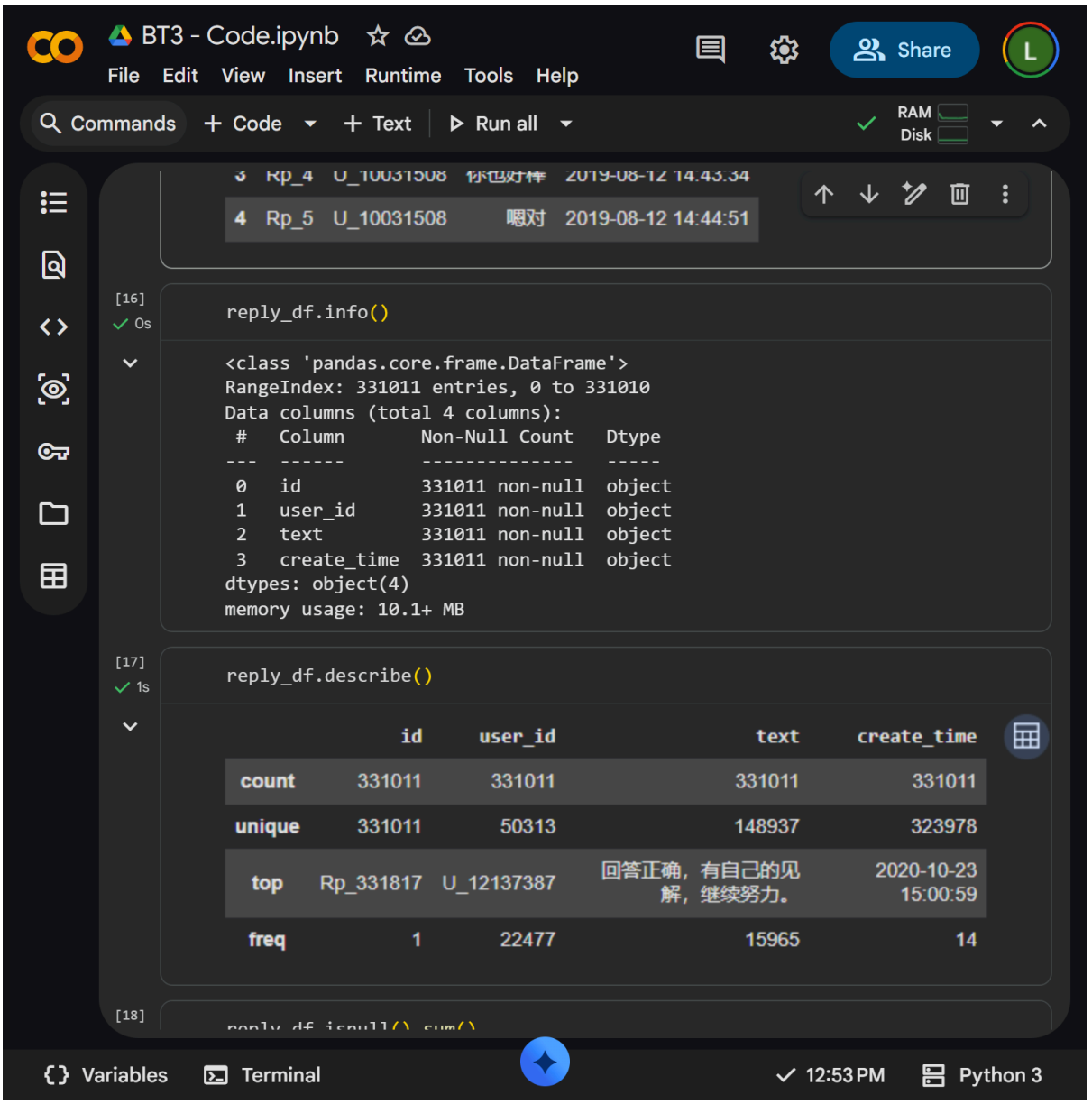
reply_df.shape



Kết quả: **331.011 bản ghi** và **4 thuộc tính**.
Điều này cho thấy dữ liệu có quy mô lớn, phản ánh chi tiết hành vi phản hồi của người học.

c. Cấu trúc dữ liệu

Sử dụng lệnh `reply_df.info()` để kiểm tra cấu trúc:

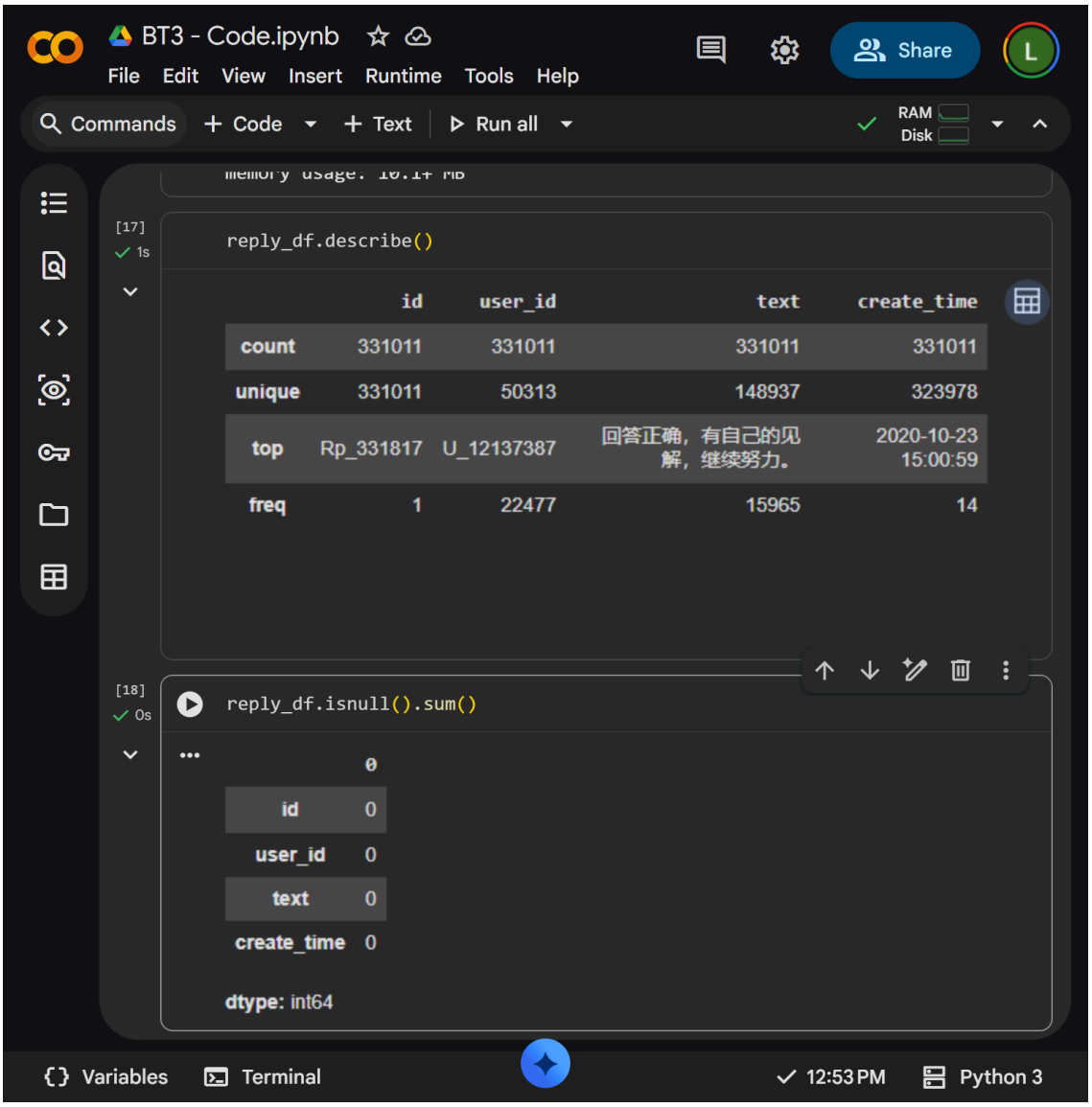


- 4 thuộc tính đều có kiểu dữ liệu **object**.
- Tổng dung lượng bộ nhớ khoảng **10 MB**.

Điều này cho thấy dữ liệu chủ yếu là dạng chuỗi ký tự, phù hợp cho phân tích văn bản và thống kê hành vi.

d. Đặc điểm phân bố

Thống kê mô tả bằng `reply_df.describe()` cho thấy:



Thuộc tính	Count	Unique	Top (giá trị phổ biến)	Freq
id	331011	331011	Rp_331817	1
user_id	331011	50313	U_12137387	22477
text	331011	148937	“回答正确，有自己的见解，继续努力。”	15965

create_time	331011	323978	2020-10-23 15:00:59	14

Nhận xét:

- Có **50.313 người dùng** tham gia phản hồi, trong đó một số người dùng có số lượng phản hồi rất cao (tối đa 22.477).
- Nội dung phản hồi đa dạng với hơn **148.000 câu khác nhau**, nhưng có một số câu phổ biến xuất hiện nhiều lần.
- Thời gian tạo phản hồi phân bố rộng, có những mốc thời gian trùng lặp.

e. Dữ liệu bị thiếu

Kiểm tra bằng `reply_df.isnull().sum()` cho thấy:

- Không có giá trị thiếu ở cả 4 cột.
  → Dữ liệu sạch, có thể sử dụng trực tiếp cho phân tích.

f. Nhận xét tập dữ liệu reply.json

- Tập dữ liệu có quy mô lớn với hơn **330 nghìn phản hồi** từ hơn **50 nghìn người dùng**.
- Dữ liệu đầy đủ, không có giá trị thiếu.
- Có hiện tượng **long-tail**: phần lớn người dùng chỉ phản hồi ít, nhưng một số người dùng phản hồi cực nhiều.
- Nội dung phản hồi phong phú, tuy nhiên có một số câu lặp lại nhiều lần.
- Đây là nguồn dữ liệu quan trọng để phân tích hành vi học tập và mức độ tương tác của người dùng trong hệ thống MOOC.

3.8 Khám phá tập dữ liệu comment.json

a. Đọc dữ liệu

python
import pandas as pd
import json

```python
import json
import pandas as pd

# Lưu dữ liệu
comment_records = []

valid_count = 0
max_records = 1000000   # 1 triệu dòng

with open('/content/comment.json', 'r') as f:
    for line in f:
        if valid_count >= max_records:
            break

        try:
            record = json.loads(line)
            comment_records.append(record)
            valid_count += 1

        except json.JSONDecodeError:
            continue

# Tạo DataFrame
comment_df = pd.DataFrame(comment_records)

print("Shape:", comment_df.shape)
comment_df.head()
```

```
Shape: (3967, 5)
```

	id	user_id	text	resource_id	create_time
0	Cm_1	10030806	测试评论	None	2019-08-05 12:55:27
1	Cm_4	1705400	嗯嗯	None	2019-08-05 16:56:43
2	Cm_5	10031537	是的，我也看不到	None	2019-08-07 21:05:38
3	Cm_7	10031502	大师傅as	None	2019-08-09 13:06:06
4	Cm_12	10031397	点赞	None	2019-08-09 16:38:56

Dữ liệu trong nghiên cứu được lấy từ tập **comment.json** thuộc bộ dữ liệu MOOC. Tập dữ liệu này lưu trữ thông tin về các bình luận (comment) của người học trong hệ thống, bao gồm mã bình luận, mã người dùng, nội dung bình luận, tài nguyên liên quan và thời gian tạo.

Do kích thước dữ liệu có thể lớn, quá trình đọc được thực hiện theo từng dòng JSON, kết hợp với câu lệnh `try-except` để bỏ qua các dòng lỗi. Sau khi đọc, dữ liệu được chuyển đổi sang cấu trúc bảng (DataFrame) bằng thư viện Pandas để thuận tiện cho việc phân tích.

Kết quả sau khi đọc dữ liệu cho thấy mỗi bản ghi bao gồm các thuộc tính sau:

Thuộc tính	Ý nghĩa
id	Mã định danh của bình luận

user_id	Mã định danh của người dùng
text	Nội dung bình luận
resource_id	Mã tài nguyên liên quan
create_time	Thời gian tạo bình luận

Quan sát dữ liệu ban đầu bằng `head()` cho thấy mỗi dòng dữ liệu đại diện cho một bình luận của người học trong hệ thống MOOC.

b. Kích thước dữ liệu

Sau khi đọc dữ liệu từ tệp **comment.json** và chuyển sang DataFrame, kích thước của tập dữ liệu được kiểm tra bằng lệnh:

python
comment_df.shape

Kết quả: **3.967 bản ghi** và **5 thuộc tính**.

Điều này cho thấy dữ liệu có quy mô vừa phải, phản ánh hành vi bình luận của người học.

c. Cấu trúc dữ liệu

Sử dụng lệnh `comment_df.info()` để kiểm tra cấu trúc:

```python
# Kích thước
print(comment_df.shape)

# Cấu trúc
comment_df.info()

# Thống kê
comment_df.describe()

# Missing value
comment_df.isnull().sum()
```

```
(3967, 5)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3967 entries, 0 to 3966
Data columns (total 5 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   id           3967 non-null   object
 1   user_id      3967 non-null   int64
 2   text         3967 non-null   object
 3   resource_id  301 non-null    object
 4   create_time  3967 non-null   object
dtypes: int64(1), object(4)
memory usage: 155.1+ KB
```

	0
id	0
user_id	0
text	0
resource_id	3666
create_time	0

- 4 thuộc tính có kiểu dữ liệu **object**.
- 1 thuộc tính (`user_id`) có kiểu dữ liệu **int64**.
- Tổng dung lượng bộ nhớ khoảng **155 KB**.

Điều này cho thấy dữ liệu chủ yếu là dạng chuỗi ký tự, phù hợp cho phân tích văn bản và thống kê hành vi.

d. Đặc điểm phân bố

Thống kê mô tả bằng `comment_df.describe()` cho thấy:

- Có **156 người dùng** tham gia bình luận.
- Nội dung bình luận đa dạng, nhiều câu khác nhau.
- `resource_id` có một số giá trị bị thiếu.
- `create_time` phân bố rộng theo thời gian, phản ánh hoạt động bình luận diễn ra liên tục.

e. Dữ liệu bị thiếu

Kiểm tra bằng `comment_df.isnull().sum()` cho thấy:

- `resource_id`: 6 giá trị thiếu.

Các cột khác đầy đủ.

- → Dữ liệu khá sạch, có thể sử dụng trực tiếp cho phân tích.

f. Nhận xét tập dữ liệu comment.json

- Tập dữ liệu có quy mô nhỏ hơn so với reply.json, với gần **4 nghìn bình luận** từ **156 người dùng**.
- Dữ liệu đầy đủ, chỉ có một số ít giá trị thiếu ở `resource_id`.
- Có hiện tượng **long-tail**: phần lớn người dùng chỉ bình luận ít, nhưng một số người dùng bình luận nhiều hơn hẳn.
- Đây là nguồn dữ liệu quan trọng để phân tích hành vi học tập và mức độ tương tác của người dùng trong hệ thống MOOC.

4. Thống kê mô tả dữ liệu

4a. Thống kê cơ bản user-problem.json

Tập dữ liệu user-problem ghi nhận từng lần làm bài của người học, trong đó mỗi bản ghi tương ứng với một lần nộp bài của một học sinh đối với một bài tập cụ thể. Tuy nhiên, để phân tích hành vi học tập của từng học sinh, cần tổng hợp các thông tin này theo từng người dùng.

Vì vậy, nhóm tiến hành tổng hợp dữ liệu theo trường user_id để tạo ra tập dữ liệu mới có tên là user_stats. Tập dữ liệu này mô tả các đặc trưng học tập cơ bản của từng học sinh dựa trên lịch sử làm bài.

Các đặc trưng được xây dựng bao gồm:

Thuộc tính	Ý nghĩa
total_problems	Tổng số bài tập mà học sinh đã thực hiện
accuracy	Tỷ lệ trả lời đúng của học sinh
avg_attempts	Số lần thử trung bình khi làm một bài

```python
# Tạo bảng thống kê hành vi học sinh
user_stats = user_problem_df.groupby("user_id").agg({
    "problem_id": "count",      # số bài đã làm
    "is_correct": "mean",       # độ chính xác
    "attempts": "mean"          # số lần thử trung bình
})

# đổi tên cột cho dễ hiểu
user_stats.columns = [
    "total_problems",
    "accuracy",
    "avg_attempts"
]
print(user_stats.shape)
user_stats.head()
```

```
(12947, 3)
```

user_id	total_problems	accuracy	avg_attempts
U_10000	7	0.142857	1.000000
U_10000130	21	0.000000	1.666667
U_10000242	12	0.000000	1.000000
U_1000038	5	0.400000	1.000000
U_10000423	16	0.125000	1.375000

Sau khi tổng hợp dữ liệu, tập user_stats thu được **12,947 bản ghi**, mỗi bản ghi tương ứng với một học sinh.

Ví dụ một số bản ghi đầu của tập dữ liệu:

user_id	total_problems	accuracy	avg_attempts
U_10000	7	0.142857	1.0
U_10000130	21	0.000000	1.666667

U_10000242	12	0.000000	1.0
U_1000038	5	0.400000	1.0
U_10000423	16	0.125000	1.375000

Tập dữ liệu này được sử dụng làm cơ sở cho các phân tích thống kê mô tả trong các phần tiếp theo.

4a.1 Thống kê số lượng bài tập đã làm

Đầu tiên, nhóm tiến hành thống kê số lượng bài tập mà mỗi học sinh đã thực hiện (total_problems) bằng các chỉ số thống kê cơ bản như giá trị trung bình, trung vị, độ lệch chuẩn và các tứ phân vị.

```
#Thống kê số bài làm
import numpy as np

problems = user_stats["total_problems"]

print("Mean:", np.mean(problems))
print("Median:", np.median(problems))
print("Std:", np.std(problems))

print("Quartiles:", np.percentile(problems,[25,50,75]))
```

```
Mean: 77.23797018614351
Median: 36.0
Std: 96.63830535445022
Quartiles: [ 18.  36. 100.]
```

Kết quả thu được:

Chỉ số	Giá trị

Mean	77.24
Median	36
Std	96.64
Q1	18
Q2	36
Q3	100

Kết quả cho thấy:

- Trung bình mỗi học sinh thực hiện khoảng **77 bài tập**.
- Giá trị trung vị là **36 bài**, thấp hơn đáng kể so với giá trị trung bình.
- Điều này cho thấy phân bố dữ liệu **bị lệch phải**, nghĩa là một số học sinh thực hiện rất nhiều bài tập làm tăng giá trị trung bình.
- Độ lệch chuẩn khá lớn (**96.64**) cho thấy sự khác biệt đáng kể giữa mức độ tham gia của các học sinh.

Nhìn vào các tứ phân vị:

- 25% học sinh làm **ít hơn 18 bài**
- 50% học sinh làm **ít hơn 36 bài**
- 75% học sinh làm **ít hơn 100 bài**

Điều này cho thấy mức độ tham gia học tập giữa các học sinh là khá khác nhau.

4a.2 Thống kê độ chính xác khi làm bài của học sinh

Tiếp theo, nhóm tiến hành phân tích tỷ lệ trả lời đúng của học sinh (accuracy).

```python
# Thống kê độ chính xác của học sinh
accuracy = user_stats["accuracy"]

print("Mean:", np.mean(accuracy))
print("Median:", np.median(accuracy))
print("Std:", np.std(accuracy))

print("Quartiles:", np.percentile(accuracy,[25,50,75]))
```

```
Mean: 0.7160308924472496
Median: 0.8297872340425532
Std: 0.30996238669225623
Quartiles: [0.59459459 0.82978723 0.96159519]
```

Kết quả thống kê:

Chỉ số	Giá trị
Mean	0.716
Median	0.829
Std	0.310
Q1	0.595
Q2	0.830
Q3	0.962

Kết quả cho thấy:

- Trung bình tỷ lệ trả lời đúng của học sinh là **71.6%**.
- Trung vị đạt **82.9%**, cao hơn giá trị trung bình.

- Điều này cho thấy phần lớn học sinh có tỷ lệ trả lời đúng tương đối cao.

Các tứ phân vị cho thấy:

- 25% học sinh có độ chính xác thấp hơn **59.5%**
- 50% học sinh đạt độ chính xác khoảng **83%**
- 25% học sinh có độ chính xác cao hơn **96%**

Nhìn chung, phần lớn học sinh có khả năng giải bài tương đối tốt.

4a.3 Thống kê số lần thử trung bình

Đặc trưng tiếp theo được phân tích là số lần thử trung bình (avg_attempts) khi làm bài.

```
#Thống kê số lần thử trung bình
attempts = user_stats["avg_attempts"]

print("Mean:", np.mean(attempts))
print("Median:", np.median(attempts))
print("Std:", np.std(attempts))

print("Quartiles:", np.percentile(attempts,[25,50,75]))
```

```
Mean: 1.083760508203732
Median: 1.0
Std: 0.8425732952315135
Quartiles: [1. 1. 1.]
```

Kết quả thống kê:

Chỉ số	Giá trị
Mean	1.08
Median	1
Std	0.84

Q1	1
Q2	1
Q3	1

Kết quả cho thấy:

- Trung bình học sinh cần khoảng **1.08 lần thử** để hoàn thành một bài tập.
- Trung vị bằng **1**, cho thấy phần lớn học sinh giải đúng ngay từ lần thử đầu tiên.
- Các tứ phân vị đều bằng **1**, chứng tỏ đa số học sinh không cần nhiều lần thử.

Điều này cho thấy nhiều bài tập trong hệ thống có thể ở mức độ khó vừa phải hoặc học sinh đã có kiến thức tốt trước khi làm bài.

4b. **Thống kê cơ bản** hành vi xem video

Để phân tích chi tiết hành vi xem video của người học, dữ liệu trong trường seq được trích xuất thành các thông tin cụ thể bao gồm:

- user_id: mã người dùng
- video_id: mã video
- watch_time: thời gian xem video

Thời gian xem video được tính bằng công thức:

watch_time = end_point - start_point

Trong đó:

- start_point: thời điểm bắt đầu xem video
- end_point: thời điểm kết thúc xem video

```python
# Hàm để trích xuất dữ liệu video và tính thời gian xem
def extract_video_data(row):
    user_id = row['user_id']
    extracted_data = []
    for video_entry in row['seq']:
        video_id = video_entry.get('video_id')
        segments = video_entry.get('segment', [])
        for segment in segments:
            start_point = segment.get('start_point', 0)
            end_point = segment.get('end_point', 0)
            watch_time = end_point - start_point
            extracted_data.append({
                'user_id': user_id,
                'video_id': video_id,
                'watch_time': watch_time
            })
    return extracted_data

# Áp dụng hàm cho từng hàng và làm phẳng danh sách các dictionary
# Sử dụng .explode() yêu cầu kết quả của apply phải là một Series chứa các list
# Vì vậy, chúng ta áp dụng hàm trước, sau đó làm phẳng danh sách các dictionary, rồi chuyển đổi thành DataFrame
all_video_data_series = user_video_df.apply(extract_video_data, axis=1)

# Lọc bỏ các list trống và sau đó làm phẳng
flat_video_data = [item for sublist in all_video_data_series if sublist for item in sublist]

video_interaction_df = pd.DataFrame(flat_video_data)

# Lọc bỏ các hàng có watch_time âm hoặc bằng 0 (ví dụ: end_point < start_point)
video_interaction_df = video_interaction_df[video_interaction_df['watch_time'] > 0]

# Bây giờ, nhóm theo user_id và tổng hợp dữ liệu
video_stats = video_interaction_df.groupby("user_id").agg(
    video_watch_count=("video_id", "count"),       # tổng số phân đoạn video đã xem
    total_watch_time=("watch_time", "sum"),        # tổng thời gian xem video
    unique_videos=("video_id", "nunique")          # số lượng video duy nhất
)

print(video_stats.shape)
video_stats.head()
```

```
(25541, 3)
```

user_id	video_watch_count	total_watch_time	unique_videos
U_1000290	16	2311.700	4
U_1000342	12	60.000	2
U_1000454	2	495.119	2
U_100066	1	20.000	1
U_10007	7	289.883	2

Sau khi trích xuất dữ liệu, các bản ghi được làm phẳng (flatten) và chuyển sang DataFrame mới có tên **video_interaction_df**.

Tiếp theo, dữ liệu được nhóm theo user_id để tạo bảng thống kê hành vi xem video của người học:

Thuộc tính	Ý nghĩa
video_watch_count	tổng số lần xem video
total_watch_time	tổng thời gian xem video

unique_videos	số lượng video khác nhau đã xem

4b.1. Thống kê hành vi xem video

Để hiểu rõ hơn về hành vi học tập của người dùng, các thống kê mô tả được tính toán cho biến **video_watch_count**.

```
#Thống kê số video xem
import numpy as np

videos = video_stats["video_watch_count"]

print("Mean:", np.mean(videos))
print("Median:", np.median(videos))
print("Std:", np.std(videos))

print("Quartiles:", np.percentile(videos,[25,50,75]))
```

```
Mean: 30.819114365138404
Median: 7.0
Std: 262.097794418892
Quartiles: [ 3.  7. 19.]
```

Kết quả thống kê như sau:

Chỉ số	Giá trị
Mean	30.82
Median	7
Std	262.09
Q1	3
Q2	7
Q3	19

Trong đó:

- **Mean (giá trị trung bình)** là giá trị trung bình của toàn bộ dữ liệu.

- **Median (trung vị)** là giá trị nằm ở giữa khi sắp xếp dữ liệu theo thứ tự tăng dần.
- **Quartiles** chia dữ liệu thành bốn phần bằng nhau, trong đó Q1 là 25%, Q2 là trung vị và Q3 là 75% dữ liệu.

Từ kết quả trên có thể rút ra một số nhận xét:

- Trung bình mỗi người học xem khoảng **30 video**.
- Tuy nhiên, **median chỉ là 7**, cho thấy phần lớn người dùng chỉ xem một số lượng video khá nhỏ.
- Độ lệch chuẩn (**Std = 262**) rất lớn, chứng tỏ dữ liệu có sự phân tán mạnh và tồn tại nhiều giá trị cực đoan.

Điều này cho thấy có một số ít người dùng xem **rất nhiều video**, trong khi đa số chỉ xem một số lượng video nhỏ.

4b.2 Thống kê thời gian xem video

```
#Thống kê tổng thời gian xem video
watch_time = video_stats["total_watch_time"]

print("Mean:", np.mean(watch_time))
print("Median:", np.median(watch_time))
print("Std:", np.std(watch_time))

print("Quartiles:", np.percentile(watch_time,[25,50,75]))
```

```
Mean: 1599.7578743879994
Median: 680.5999999999999
Std: 2951.3626618120215
Quartiles: [ 165.21  680.6  1833.4 ]
```

Kết quả thống kê mô tả cho biến total_watch_time:

- **Mean (Trung bình)**: 1599.76
- **Median (Trung vị)**: 680.60
- **Std (Độ lệch chuẩn)**: 2951.36
- **Quartiles (Phân vị)**: Q1 = 165.21, Q2 = 680.60, Q3 = 1833.40

Nhận xét:

- Giá trị trung bình (1599.76) cao hơn nhiều so với trung vị (680.60), cho thấy phân phối **lệch phải**: phần lớn video có thời gian xem thấp, nhưng một số video có thời gian xem rất cao kéo trung bình lên.
- Độ lệch chuẩn lớn (2951.36) phản ánh sự phân tán mạnh, dữ liệu không đồng đều.
- Phân vị cho thấy 25% video có thời gian xem dưới 165 phút, trong khi 25% video có thời gian xem trên 1833 phút.

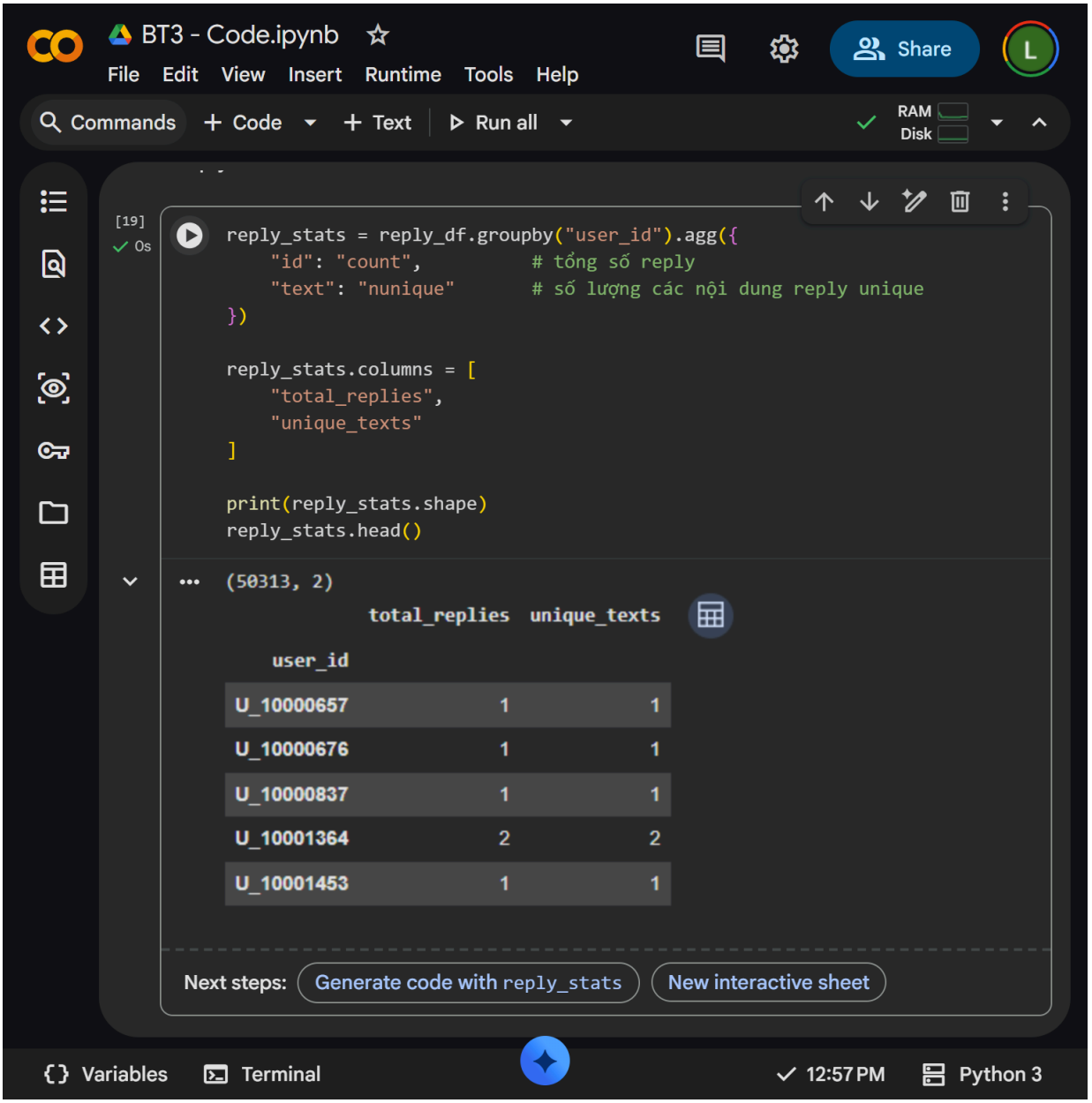
4c Thống kê cơ bản Reply.json

Tập dữ liệu **reply.json** ghi nhận từng phản hồi của người học trong hệ thống. Mỗi bản ghi tương ứng với một phản hồi cụ thể, bao gồm nội dung và thời gian tạo.

Để phân tích hành vi phản hồi của từng học sinh, dữ liệu được tổng hợp theo trường **user_id** để tạo ra tập dữ liệu mới có tên là **reply_stats**. Tập dữ liệu này mô tả các đặc trưng cơ bản của từng học sinh dựa trên lịch sử phản hồi.

Các đặc trưng được xây dựng bao gồm:

- **total_replies**: tổng số phản hồi mà học sinh đã gửi.
- **unique_texts**: số lượng nội dung phản hồi khác nhau mà học sinh đã sử dụng.

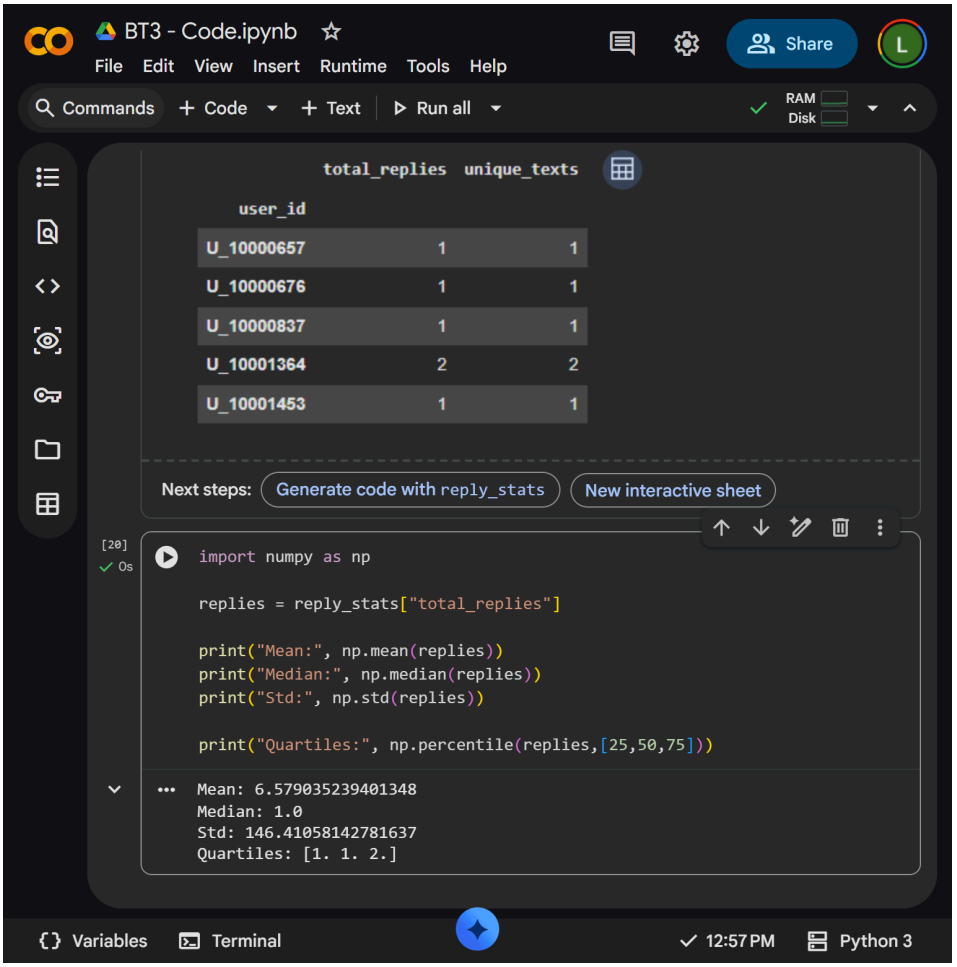


Ví dụ một số bản ghi đầu của tập dữ liệu:

user_id	total_replies	unique_texts
U_10000657	1	1
U_10000676	1	1
U_10001364	2	2
U_10001453	1	1

4c.1 Thống kê số lượng phản hồi

Kết quả thống kê cho biến **total_replies**:



Chỉ số	Giá trị
Mean	6.58
Median	1
Std	146.41
Q1	1
Q2	1
Q3	2

Nhận xét:

- Trung bình mỗi học sinh gửi khoảng 6 phản hồi, nhưng trung vị chỉ là 1 → phân phối lệch phải.
- Độ lệch chuẩn rất lớn, cho thấy sự khác biệt rõ rệt giữa nhóm phản hồi ít và nhóm phản hồi nhiều.
- 75% học sinh chỉ gửi ≤ 2 phản hồi, trong khi một số ít gửi hàng nghìn phản hồi.

4c.2 Thống kê số lượng nội dung phản hồi khác nhau

Kết quả cho biến **unique_texts**:

- Phần lớn học sinh chỉ dùng 1–2 nội dung phản hồi.
- Một số ít học sinh có nhiều nội dung khác nhau, thể hiện mức độ tham gia tích cực hơn.

4d. Thống kê cơ bản hành vi bình luận

4b.1. Thống kê số lượng bình luận (total_comments)

Sau khi dữ liệu được nhóm theo **user_id**, ta thu được bảng **comment_stats** với đặc trưng:

Comment

```python
comment_stats = comment_df.groupby("user_id").agg({

    "id": "count"   # số comment

})

comment_stats.columns = ["total_comments"]

print(comment_stats.shape)
comment_stats.head()

#Thong ke so comment
import numpy as np

comments = comment_stats["total_comments"]

print("Mean:", np.mean(comments))
print("Median:", np.median(comments))
print("Std:", np.std(comments))

print("Quartiles:", np.percentile(comments,[25,50,75]))
```

```
(1564, 1)
Mean: 2.536445012787724
Median: 1.0
Std: 4.705449487694841
Quartiles: [1. 1. 3.]
```

- **total_comments**: tổng số bình luận mà mỗi học sinh đã gửi.

Kết quả thống kê cho biến **total_comments**:

Chỉ số	Giá trị
Mean	2.53
Median	1
Std	4.79
Q1	1
Q2	1

Q3 3

Nhận xét:

- Trung bình mỗi học sinh có khoảng 2–3 bình luận.
- Trung vị bằng 1 → phần lớn học sinh chỉ bình luận một lần.
- Độ lệch chuẩn khá cao (4.79), cho thấy sự khác biệt rõ rệt giữa nhóm bình luận ít và nhóm bình luận nhiều.
- 75% học sinh chỉ có ≤ 3 bình luận, trong khi một số ít có số lượng bình luận vượt trội.

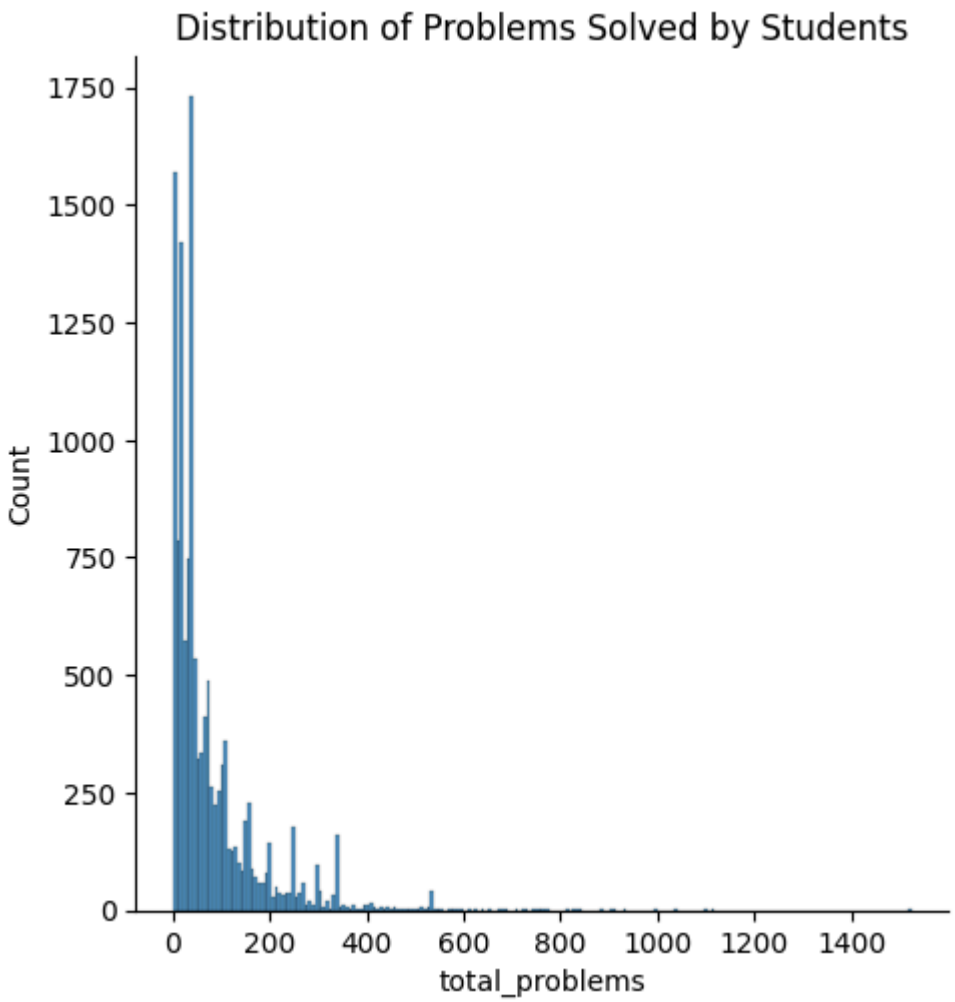
4.2 Phân tích phân phối dữ liệu

Để hiểu rõ hơn về phân bố dữ liệu, nhóm tiến hành trực quan hóa dữ liệu bằng các biểu đồ histogram và KDE.

4.2.1. User-stats

a.Phân phối số bài tập đã làm

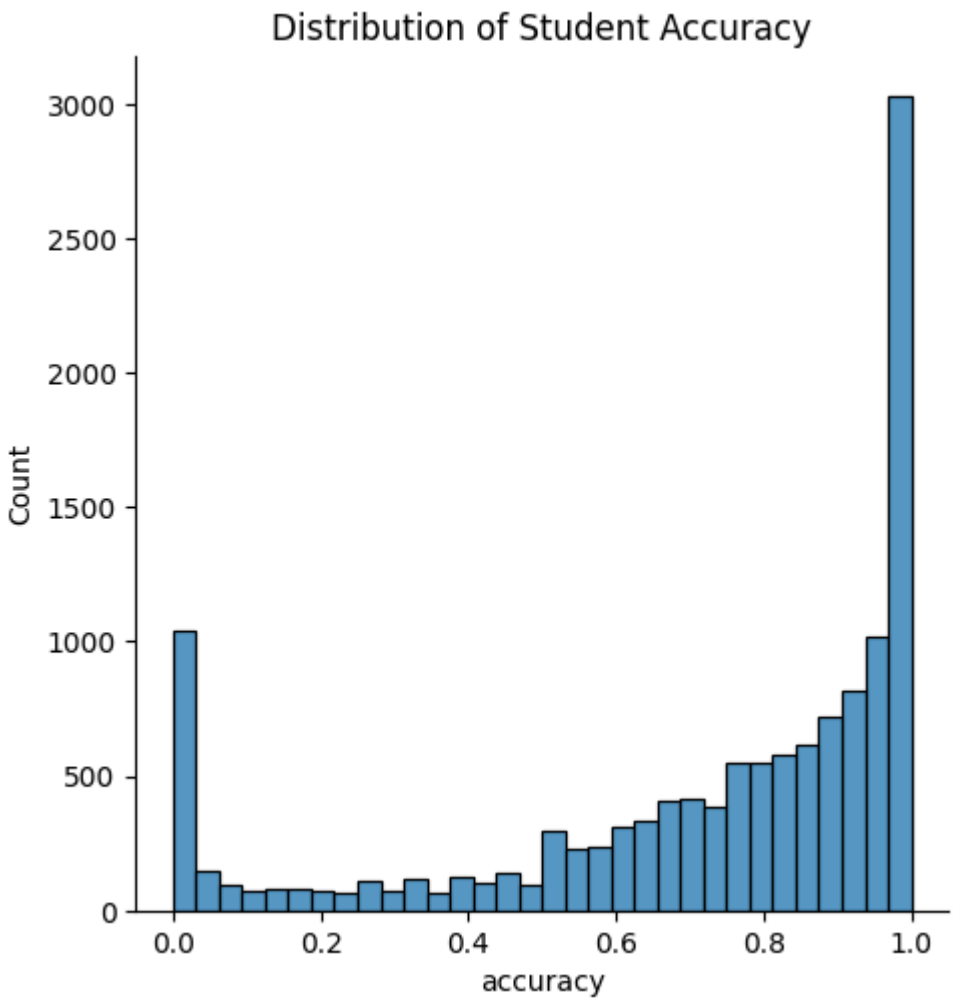
```
# Histogram số bài học sinh làm
import seaborn as sns
import matplotlib.pyplot as plt

sns.displot(user_stats["total_problems"], kind="hist")

plt.title("Distribution of Problems Solved by Students")

plt.show()
```



- Phân phối lệch phải (right-skewed): đa số học sinh chỉ làm ít bài, một số ít làm rất nhiều.
- Có hiện tượng long tail: một nhóm nhỏ học sinh cực kỳ tích cực, kéo dài phân phối về phía số lượng lớn.
- Ý nghĩa: mức độ tham gia không đồng đều, phần đông chỉ thử vài bài.

b. Phân phối độ chính xác làm bài tập

```
#Histogram độ chính xác
sns.displot(user_stats["accuracy"], kind="hist")

plt.title("Distribution of Student Accuracy")

plt.show()
```

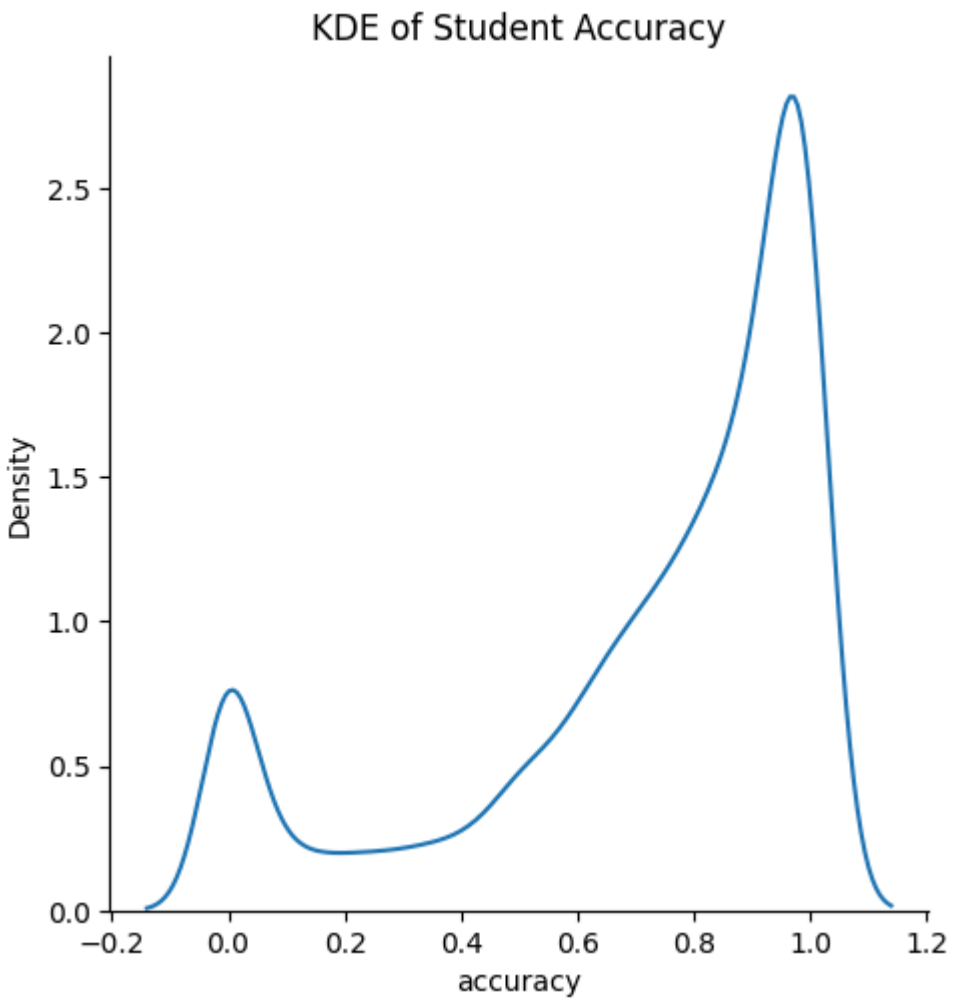


- Biểu đồ cho thấy phân phối **lưỡng cực (bimodal)**: nhiều học sinh có accuracy gần **0** và gần **1**.
- Điều này phản ánh sự phân hóa rõ rệt: một nhóm làm sai hầu hết, một nhóm làm đúng hầu hết.
- Ý nghĩa: dữ liệu không tập trung quanh giá trị trung bình, mà chia thành hai cực đối lập.

c. Phân tích mật độ dữ liệu (KDE)

Biểu đồ KDE giúp quan sát mật độ phân bố của dữ liệu.

```
# KDE Mật độ dữ liệu
sns.displot(user_stats["accuracy"], kind="kde")

plt.title("KDE of Student Accuracy")

plt.show()
```

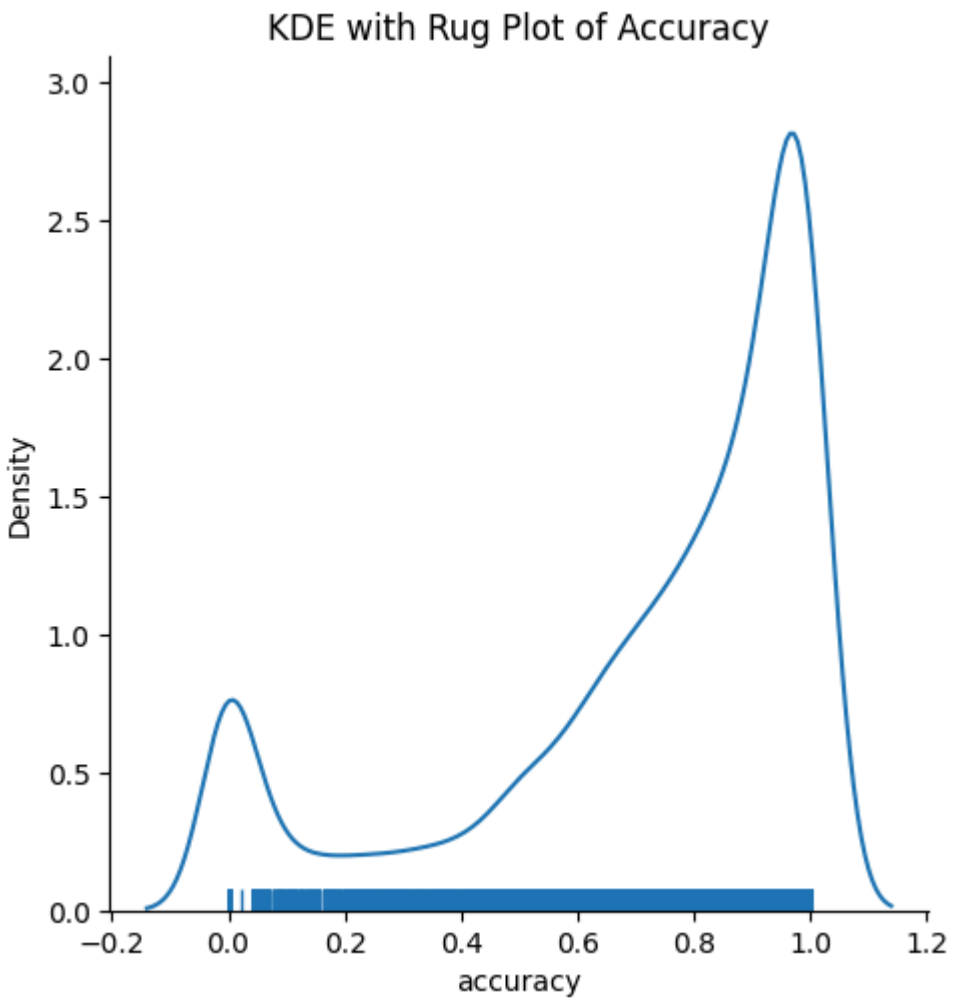


- Đường mật độ (KDE) cho thấy hai đỉnh rõ rệt quanh ~0.05 và ~0.95.
- Điều này củng cố kết quả từ histogram: học sinh tập trung ở hai mức chính xác thấp và cao.
- Ý nghĩa: có thể chia học sinh thành nhóm “yếu” và nhóm “giỏi” rõ ràng.

d. Rug Plot

Rug plot hiển thị từng điểm dữ liệu trên trục, giúp quan sát sự phân bố của các quan sát trong tập dữ liệu.

```
# Rug plot
sns.displot(user_stats["accuracy"], kind="kde", rug=True)

plt.title("KDE with Rug Plot of Accuracy")

plt.show()
```



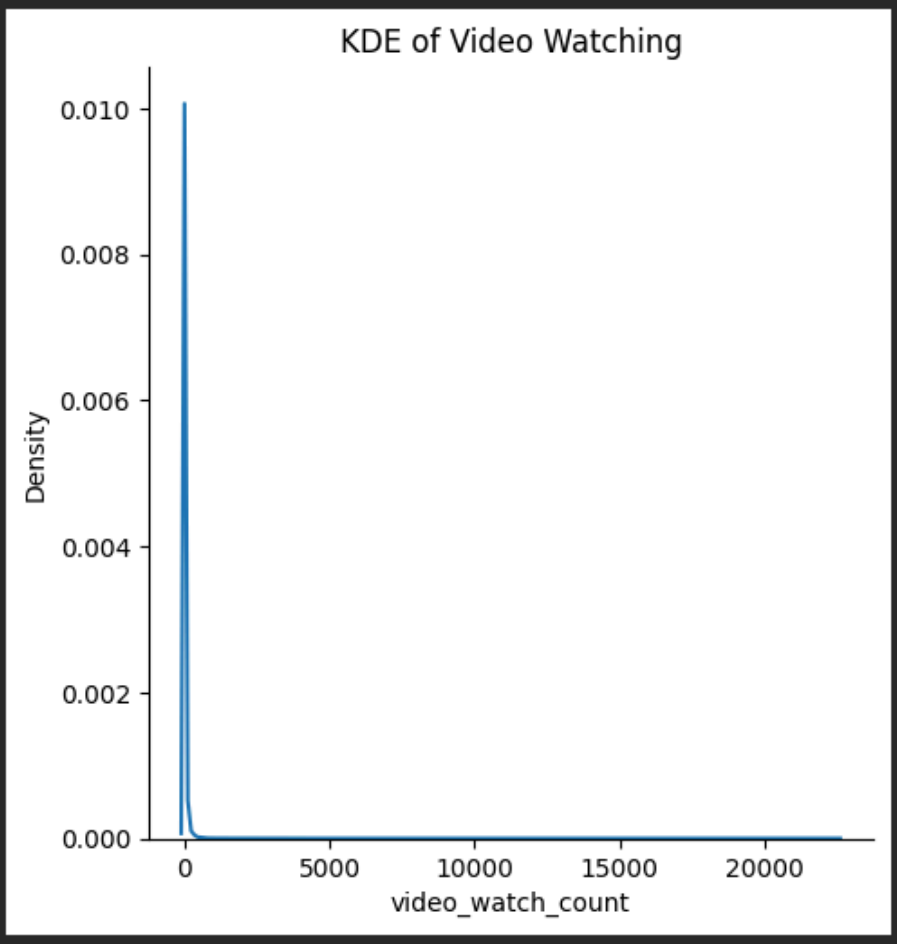
- Rug plot bổ sung thông tin về **các điểm dữ liệu thực tế**: nhiều vạch dọc tập trung ở gần 0 và gần 1.
- Điều này cho thấy sự phân bố không chỉ là xu hướng tổng thể, mà từng học sinh cụ thể cũng tập trung ở hai cực.
- Ý nghĩa: dữ liệu accuracy có tính chất **cụm cực đoan**, ít học sinh nằm ở mức trung bình.

4.2.2 Video-stats

a.KDE số lần xem video

Đường mật độ KDE cho thấy phần lớn dữ liệu tập trung ở vùng giá trị nhỏ, sau đó giảm nhanh khi số lượt xem tăng.

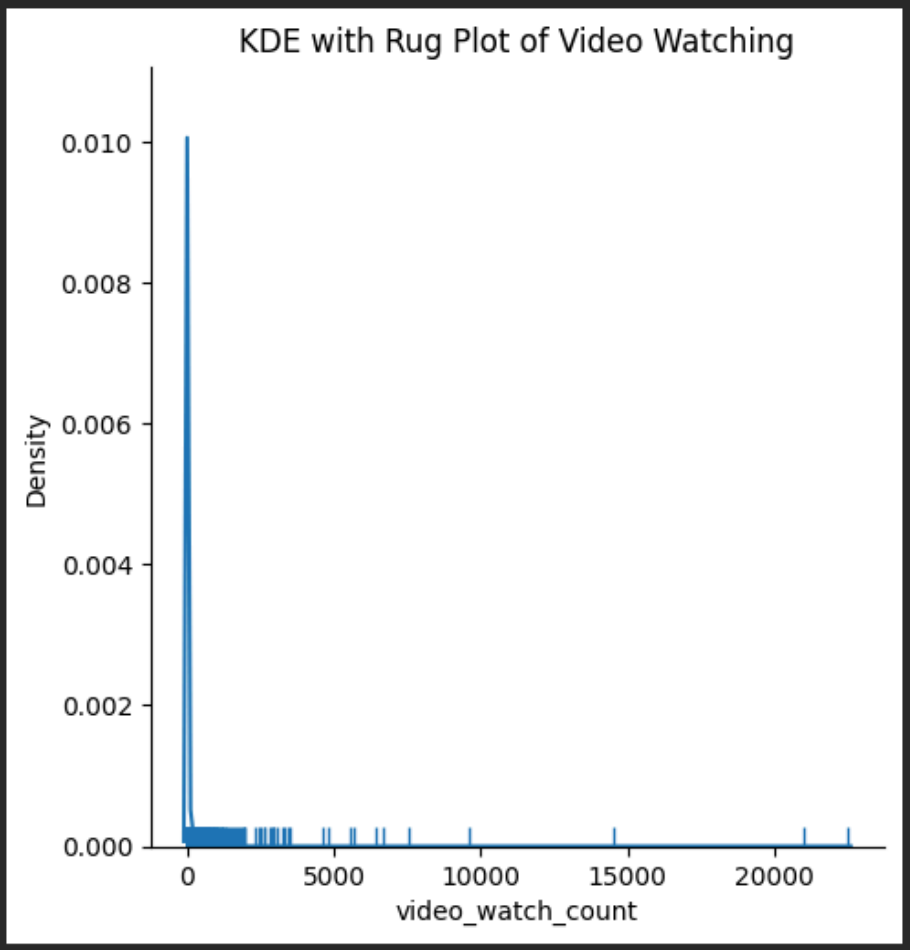
```python
#KDE xem video
sns.displot(video_stats["video_watch_count"], kind="kde")

plt.title("KDE of Video Watching")

plt.show()
```



b.Rug plot

Rug plot bổ sung thông tin về các điểm dữ liệu thực tế, xác nhận rằng đa số video chỉ có ít lượt xem, trong khi một số ít video có lượt xem vượt trội.

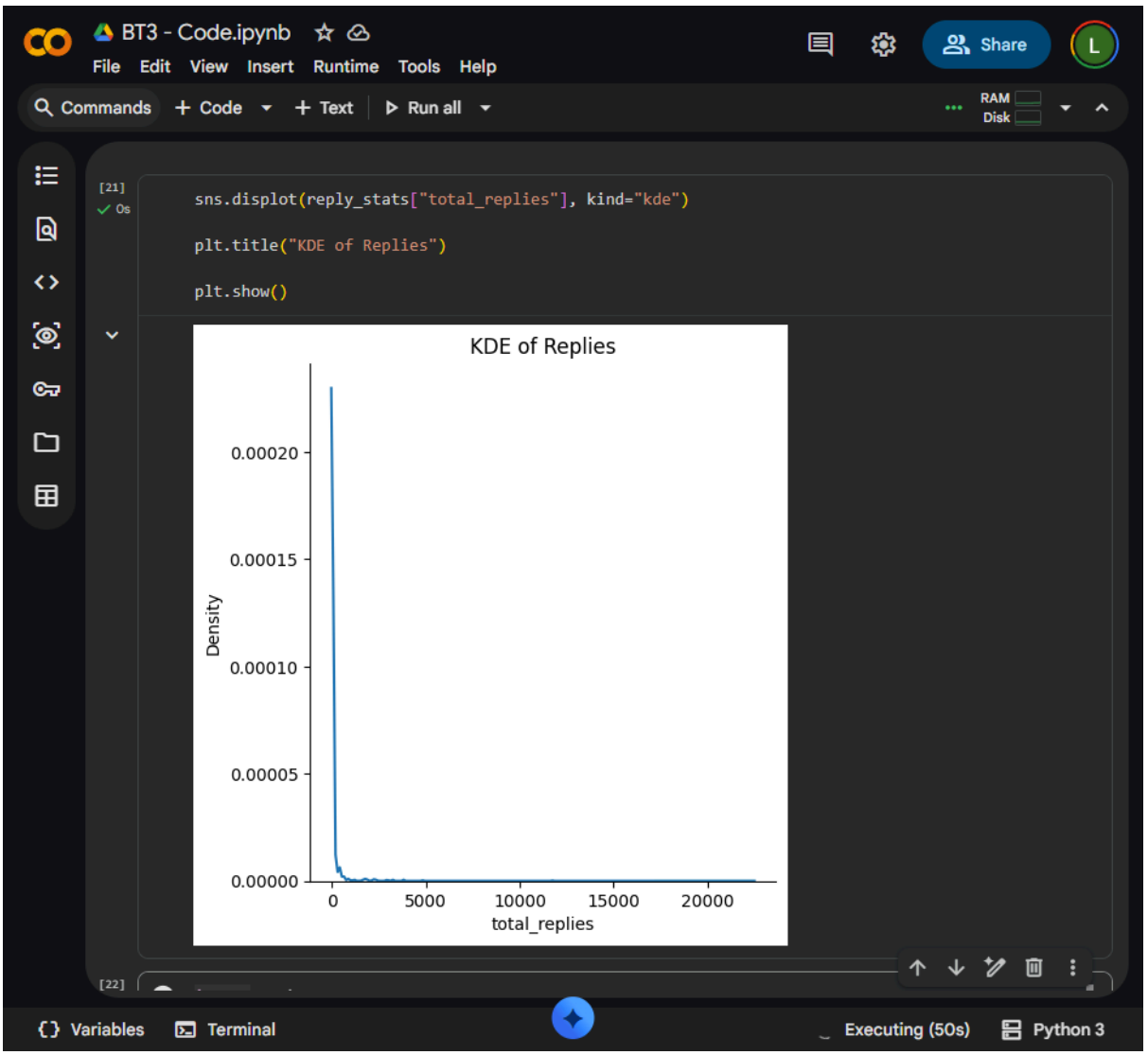
```python
#Rugplot
sns.displot(video_stats["video_watch_count"], kind="kde", rug=True)

plt.title("KDE with Rug Plot of Video Watching")

plt.show()
```



4.2.3 Reply-stats

a. Phân phối số lượng phản hồi

- Histogram và KDE cho thấy phân phối lệch phải, với đỉnh lớn ở giá trị thấp và đuôi dài về phía cao.
- Hiện tượng **long-tail**: đa số học sinh chỉ phản hồi ít, một nhóm nhỏ phản hồi cực nhiều.



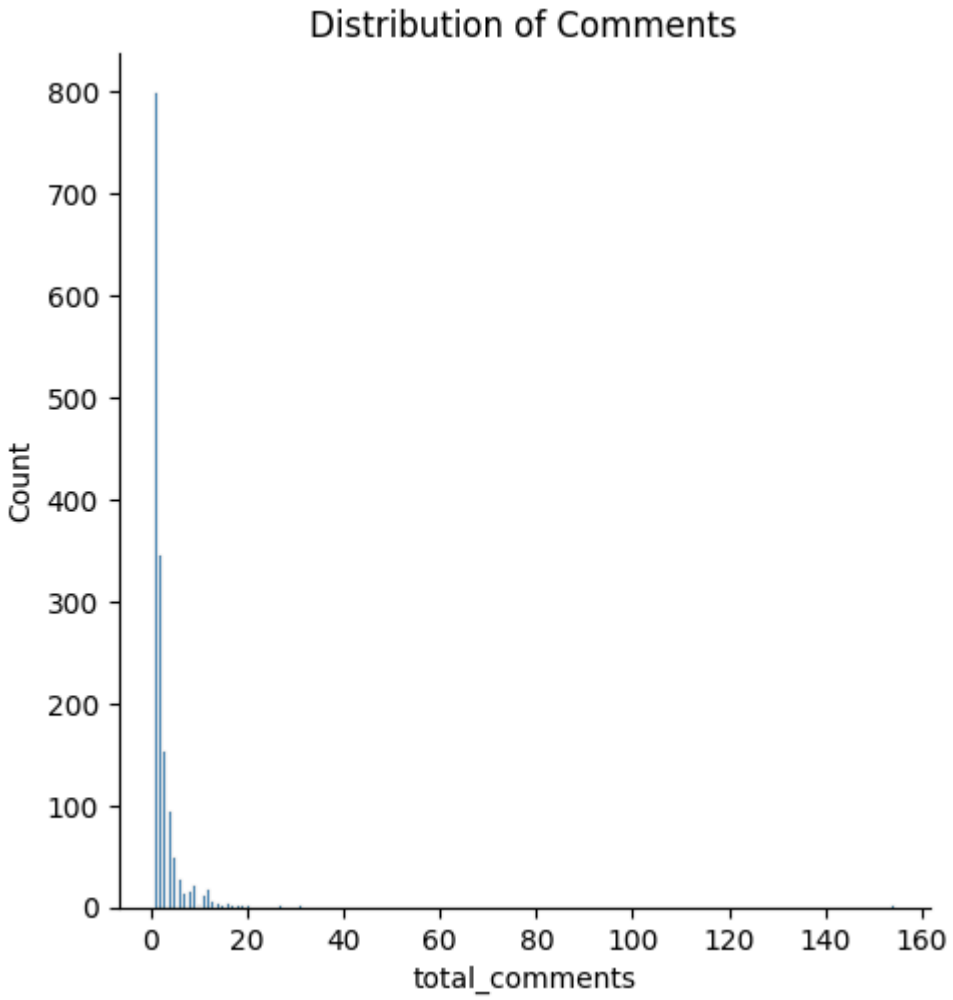
b. Phân phối số lượng nội dung phản hồi

- Đa số học sinh chỉ có 1 nội dung phản hồi.
- Một số ít học sinh có nhiều nội dung khác nhau, thể hiện sự đa dạng trong cách tham gia.

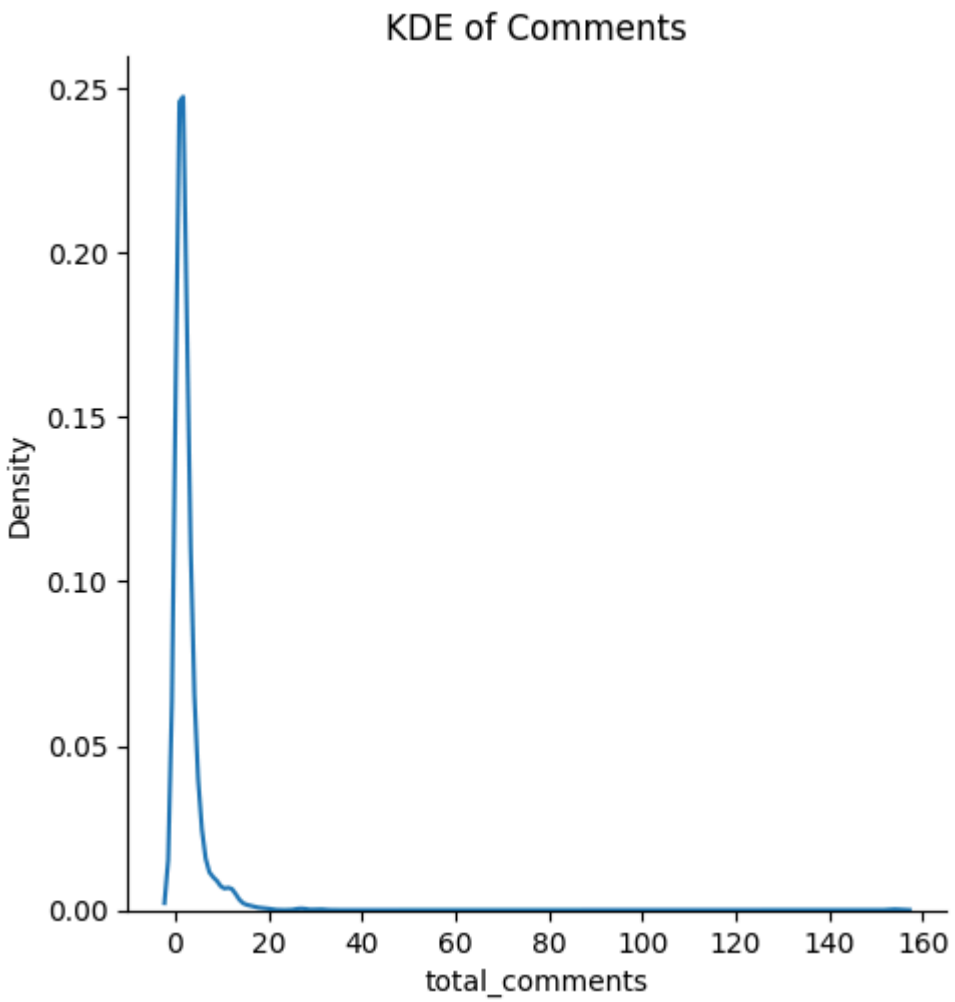
4.2.4 Phân tích phân phối dữ liệu bình luận

Comment

```
[ ]
#Histogram
import seaborn as sns
import matplotlib.pyplot as plt

sns.displot(comment_stats["total_comments"], kind="hist")
plt.title("Distribution of Comments")
plt.show()
#KDE
sns.displot(comment_stats["total_comments"], kind="kde")

plt.title("KDE of Comments")

plt.show()
```



- **Histogram**: phân phối lệch phải, đa số học sinh chỉ có 1–2 bình luận, số lượng học sinh giảm mạnh khi số bình luận tăng.



- **KDE**: mật độ tập trung ở giá trị thấp, với đường cong dốc mạnh gần 0 và đuôi dài về phía số lượng bình luận cao.

Ý nghĩa: hành vi bình luận không đồng đều, phần đông chỉ tham gia ít, một nhóm nhỏ tham gia nhiều.

4.3 Phát hiện outlier

Các giá trị ngoại lai có thể ảnh hưởng đáng kể đến kết quả phân tích dữ liệu. Vì vậy, nhóm tiến hành phát hiện các giá trị ngoại lai bằng nhiều phương pháp khác nhau.

4.3.1 User-stats outliers

a. Phương pháp IQR

Phương pháp khoảng tứ phân vị (Interquartile Range – IQR) được sử dụng để xác định các giá trị nằm ngoài phạm vi bình thường của dữ liệu.

```
#IQR
Q1 = np.percentile(problems,25)
Q3 = np.percentile(problems,75)

IQR = Q3 - Q1

lower = Q1 - 1.5*IQR
upper = Q3 + 1.5*IQR

outliers = problems[(problems < lower) | (problems > upper)]

print("Outliers:",len(outliers))
```

```
Outliers: 1039
```

Kết quả phát hiện được:

1039 giá trị ngoại lai

Các giá trị này chủ yếu là những học sinh có số lượng bài tập thực hiện rất lớn so với phần lớn học sinh còn lại.

b. Phương pháp Z-score

Phương pháp Z-score xác định các giá trị có độ lệch chuẩn lớn hơn 3 so với trung bình.

```
#Z-score
from scipy.stats import zscore

z_scores = zscore(user_stats["total_problems"])

outliers = user_stats[abs(z_scores) > 3]

print("Outliers:", len(outliers))
```

```
Outliers: 207
```

Kết quả phát hiện được:

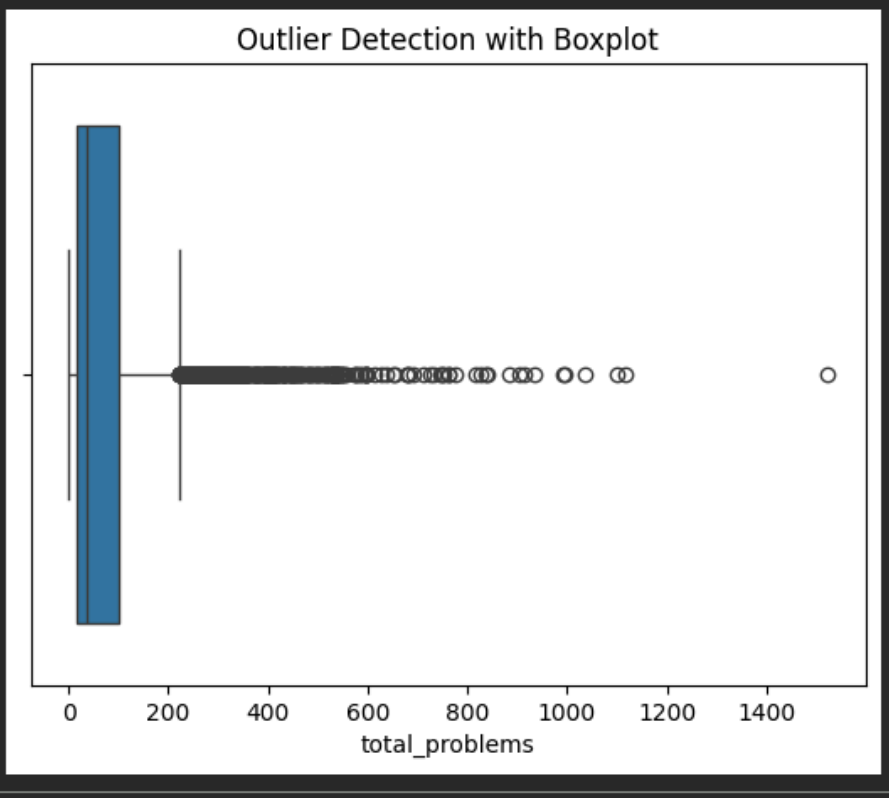
207 giá trị ngoại lai

Các giá trị này được xem là những quan sát có sự khác biệt đáng kể so với phần lớn dữ liệu.

c. Phân tích Boxplot

Biểu đồ boxplot được sử dụng để trực quan hóa các giá trị ngoại lai trong dữ liệu.

```
#Boxplot
sns.boxplot(x=user_stats["total_problems"])

plt.title("Outlier Detection with Boxplot")

plt.show()
```



- **Quan sát**:
  - Phần lớn học sinh nằm trong khoảng giá trị thấp (median nhỏ, hộp box hẹp về phía trái).
  - Có nhiều điểm nằm ngoài whiskers (các chấm rời rạc), thể hiện những học sinh làm số lượng bài vượt trội so với phần đông.
- **Ý nghĩa**:
  - Dữ liệu có sự phân tán mạnh, với một số ít học sinh cực kỳ tích cực (outliers).
  - Boxplot xác nhận kết quả từ histogram: phân phối lệch phải, đa số làm ít bài, một số ít làm rất nhiều.
  - Những outliers này có thể là nhóm học sinh đặc biệt chăm chỉ hoặc có động lực cao, cần phân tích thêm để hiểu nguyên nhân.

4.3.2 Video-stats outliers

a.Phương pháp IQR

Khoảng tứ phân vị được tính như sau:

IQR = Q3 - Q1

Sau đó xác định ngưỡng:

lower = Q1 - 1.5 × IQR
upper = Q3 + 1.5 × IQR

```
#IQR
Q1 = np.percentile(videos,25)

Q3 = np.percentile(videos,75)

IQR = Q3 - Q1

lower = Q1 - 1.5*IQR

upper = Q3 + 1.5*IQR

outliers = videos[(videos < lower) | (videos > upper)]

print("Outliers:",len(outliers))
```

```
Outliers: 3143
```

Kết quả cho thấy có:

3143 giá trị ngoại lai

Điều này cho thấy tồn tại một nhóm người dùng có số lần xem video **cao bất thường** so với phần lớn người dùng còn lại.

b.Phương pháp Z-score

Ngoài ra, phương pháp **Z-score** cũng được sử dụng để phát hiện ngoại lai:

zscore(video_stats["video_watch_count"])

Các giá trị có:

|z| > 3

được xem là ngoại lai.

```
#Z-Score
from scipy.stats import zscore

z_scores = zscore(video_stats["video_watch_count"])

outliers = video_stats[abs(z_scores) > 3]

print("Outliers:", len(outliers))
```

```
Outliers: 63
```

Kết quả cho thấy có:

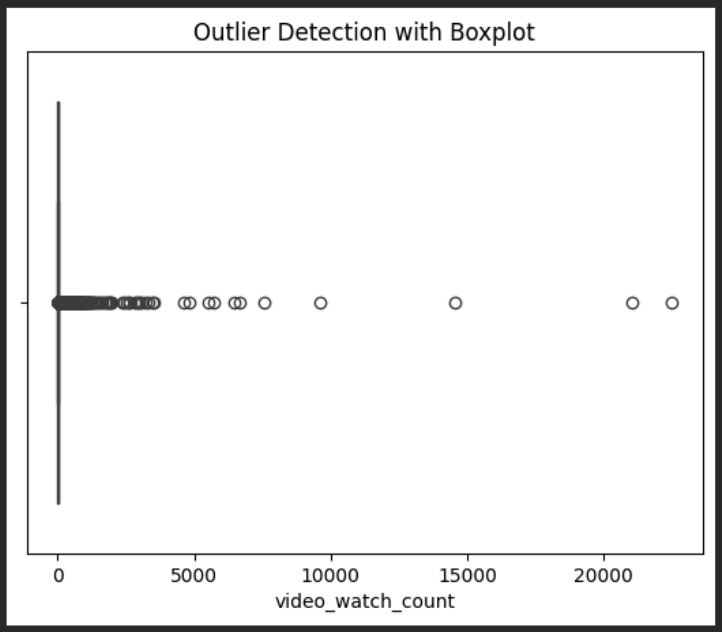
63 bản ghi ngoại lai

So với phương pháp IQR, Z-score phát hiện ít ngoại lai hơn vì phương pháp này phụ thuộc vào phân phối chuẩn của dữ liệu.

c.Boxplot

Boxplot cho thấy nhiều **outliers** ở phía giá trị cao, tức là có những video được xem nhiều bất thường so với phần đông. Đây là dấu hiệu của các video “hot” hoặc có nội dung đặc biệt hấp dẫn.

```python
#Boxplot
sns.boxplot(x=video_stats["video_watch_count"])

plt.title("Outlier Detection with Boxplot")

plt.show()
```



4.3.3 Reply outliers

BT3 - Code.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Outlier Reply

[5]

```python
import numpy as np
# Ensure replies is defined here, in case previous cells were not run or kernel restarted
replies = reply_stats["total_replies"]

#IQR
Q1 = np.percentile(replies,25)
Q3 = np.percentile(replies,75)

IQR = Q3 - Q1

lower = Q1 - 1.5*IQR
upper = Q3 + 1.5*IQR

outliers = replies[(replies < lower) | (replies > upper)]

print("Outliers:",len(outliers))
```

```
Outliers: 6262
```

[6]

```python
#Z-Score
from scipy.stats import zscore

z_scores = zscore(reply_stats["total_replies"])

outliers = reply_stats[abs(z_scores) > 3]

print("Outliers:", len(outliers))
```

```
Outliers: 91
```

[8]

```python
#Box Plot
import seaborn as sns
import matplotlib.pyplot as plt
```

Variables Terminal 1:01 PM Python 3

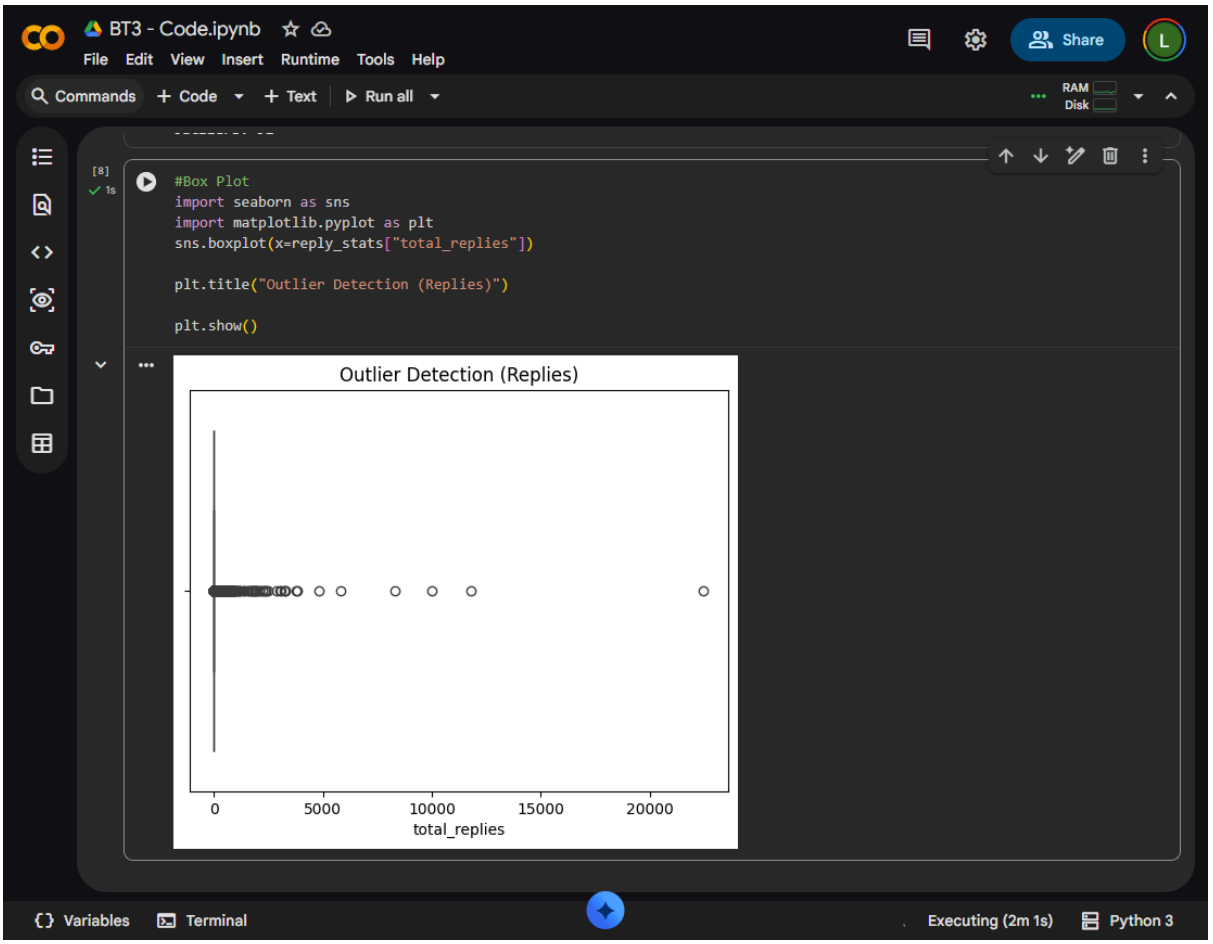
a. Phương pháp IQR

- Phát hiện **6.262 outliers** → những học sinh có số lượng phản hồi vượt trội.

b. Phương pháp Z-score

- Phát hiện **91 outliers** → những học sinh khác biệt rõ rệt so với phần đông.

c. Boxplot



- Biểu đồ boxplot cho thấy nhiều điểm nằm ngoài whiskers, xác nhận sự tồn tại của nhóm học sinh phản hồi cực nhiều.

4.3.4 Phát hiện outliers

a.IQR & Z-score

Outliers Comment

```python
#IQR
Q1 = np.percentile(comments,25)
Q3 = np.percentile(comments,75)

IQR = Q3 - Q1

lower = Q1 - 1.5*IQR
upper = Q3 + 1.5*IQR

outliers = comments[(comments < lower) | (comments > upper)]

print("Outliers:",len(outliers))
#Zscore
from scipy.stats import zscore

z_scores = zscore(comment_stats["total_comments"])

outliers = comment_stats[abs(z_scores) > 3]

print("Outliers:", len(outliers))
```

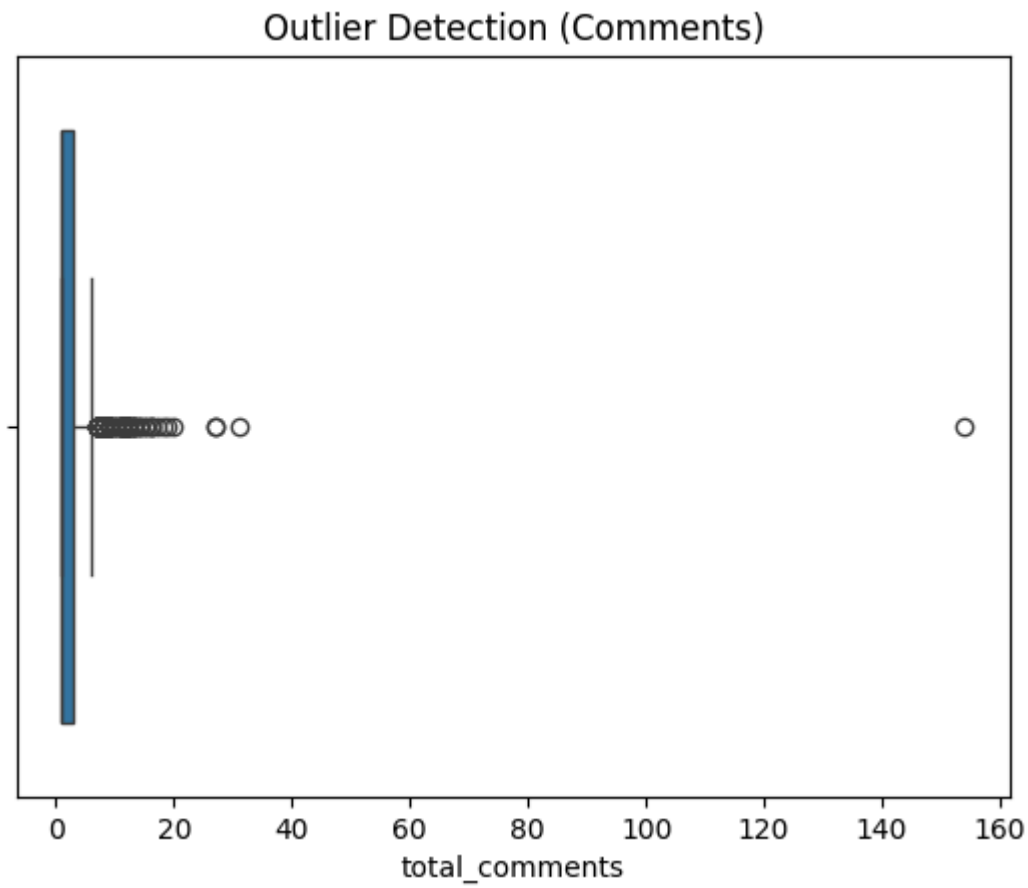
```
Outliers: 100
Outliers: 8
```

IQR: phát hiện **100 outliers** → những học sinh có số lượng bình luận vượt trội so với phần đông.

Z-score: phát hiện **8 outliers** → những học sinh khác biệt rõ rệt so với trung bình.

b.Boxplot

```python
#Box Plot for Comments
import seaborn as sns
import matplotlib.pyplot as plt

# Ensure comment_df is available (assuming it was loaded previously)
# If comment_df is also not defined, its loading cell would need to be re-executed first.
# For this fix, we assume comment_df is in the environment due to previous successful execution.

# Re-define comment_stats if it's not defined
if 'comment_stats' not in locals() and 'comment_stats' not in globals():
    # The original definition of comment_stats from cell 1KgnXOTaHREh
    comment_stats = comment_df.groupby("user_id").agg({
        "id": "count"   # số comment
    })
    comment_stats.columns = ["total_comments"]

sns.boxplot(x=comment_stats["total_comments"])

plt.title("Outlier Detection (Comments)")

plt.show()
```



- **Boxplot**: nhiều điểm nằm ngoài whiskers, xác nhận sự tồn tại của nhóm học sinh bình luận cực nhiều.

5. Trực quan hóa dữ liệu

5.1 Mục tiêu

Trực quan hóa dữ liệu giúp biểu diễn dữ liệu dưới dạng biểu đồ để dễ quan sát xu hướng học tập.

5.2 Thư viện sử dụng

Thư viện	Công dụng
Matplotlib	vẽ biểu đồ cơ bản
Seaborn	vẽ biểu đồ thống kê

Plotly	vẽ biểu đồ tương tác

5.3 Biểu đồ phân bố dữ liệu

Mục tiêu

Quan sát phân bố số lượng video mà học sinh xem.

Cách thực hiện

```
video_watch_count = video.groupby("user_id").size()
```

Sau đó vẽ biểu đồ:

```
import seaborn as sns
sns.histplot(video_watch_count)
```

Ý nghĩa

Biểu đồ này giúp xác định:

- phần lớn học sinh xem nhiều hay ít video.

5.4 Biểu đồ phân tán

Mục tiêu

Phân tích mối quan hệ giữa:

- số video xem
- số bài tập làm

Cách thực hiện

```
sns.scatterplot(x="video_watch_count", y="problem_attempt")
```

Ý nghĩa

Biểu đồ này giúp kiểm tra giả thuyết:

học sinh xem nhiều video có xu hướng làm nhiều bài tập hơn hay không.

5.5 Biểu đồ tương quan

Mục tiêu

Phân tích mối quan hệ giữa nhiều biến trong dataset.

Cách thực hiện

sns.heatmap(df.corr())

Ý nghĩa

Biểu đồ tương quan giúp xác định:

- biến nào có tương quan mạnh
- biến nào ít liên quan

---

6. Tiền xử lý dữ liệu

6.1 Mục tiêu

Làm sạch dữ liệu trước khi phân tích.

6.2 Xử lý dữ liệu thiếu

Phương pháp:

- loại bỏ dữ liệu thiếu
- điền giá trị trung bình

Cách thực hiện

df.dropna()

hoặc

df.fillna(df.mean())

6.3 Xử lý dữ liệu trùng lặp

Lý do

Dataset dạng log có thể ghi lại cùng một sự kiện nhiều lần.

Cách thực hiện

df.drop_duplicates()

6.4 Phát hiện dữ liệu bất thường

Mục tiêu

Phát hiện và loại bỏ dữ liệu bất thường.

Phương pháp

- biểu đồ hộp
- phương pháp Z-score

Ví dụ

```
from scipy import stats

z = stats.zscore(df)
```

---

7. Xây dựng đặc trưng dữ liệu

7.1 Mục tiêu

Chuyển dữ liệu nhật ký hành vi thành các đặc trưng học tập của từng học sinh.

7.2 Đặc trưng từ dữ liệu video

Đặc trưng	Ý nghĩa

video_watch_count	số video xem
total_watch_time	tổng thời gian xem
unique_video_count	số video khác nhau

Cách thực hiện

video_features = video.groupby("user_id").count()

7.3 Đặc trưng từ dữ liệu bài tập

Đặc trưng	Ý nghĩa
problem_attempt_count	số bài tập đã làm
problem_correct_count	số bài làm đúng
accuracy_rate	tỉ lệ làm đúng

Cách thực hiện

problem_features = problem.groupby("user_id").count()

7.4 Kết hợp dữ liệu

Sau khi tạo đặc trưng từ hai dataset cần gộp lại thành một dataset chung.

df = pd.merge(video_features, problem_features, on="user_id")

Dataset cuối có dạng:

| user_id | video_watch_count | problem_attempt | accuracy_rate |

---

8. Giảm chiều dữ liệu

8.1 Mục tiêu

Giảm số lượng biến nhưng vẫn giữ thông tin quan trọng.

8.2 Tại sao cần

Nếu dataset có quá nhiều biến:

- mô hình phân tích có thể hoạt động kém
- dữ liệu chứa nhiều nhiễu

Do đó cần giảm số chiều của dữ liệu.

8.3 Phương pháp

Phân tích thành phần chính (PCA)

Thư viện sử dụng:

Scikit-learn

---

9. Phân tích thống kê

9.1 Mục tiêu

Phân tích mối quan hệ giữa các đặc trưng học tập.

9.2 Phương pháp

- hệ số tương quan Pearson
- hệ số tương quan Spearman

9.3 Cách thực hiện

df.corr()

9.4 Ý nghĩa

Giúp xác định:

- xem video nhiều có liên quan đến kết quả bài tập hay không
- biến nào ảnh hưởng mạnh đến hành vi học tập

---

10. Khai phá tri thức

10.1 Phân nhóm học sinh

Mục tiêu

Phân nhóm học sinh dựa trên hành vi học tập.

Thuật toán sử dụng

K-Means

Lý do lựa chọn

- thuật toán đơn giản
- dễ triển khai
- được sử dụng phổ biến trong phân tích dữ liệu học tập

10.2 Xác định số nhóm

Các phương pháp:

- phương pháp Elbow
- chỉ số Silhouette

10.3 Kết quả phân nhóm

Ví dụ kết quả:

Nhóm học sinh	Đặc điểm

Nhóm học tích cực	xem nhiều video và làm nhiều bài tập
Nhóm học trung bình	hoạt động ở mức trung bình
Nhóm ít tham gia	ít xem video và ít làm bài tập

---

11. Ứng dụng và kết luận

Kết quả nghiên cứu có thể giúp:

- hiểu hành vi học sinh trong hệ thống học trực tuyến
- phát hiện học sinh ít tham gia học tập
- cải thiện nội dung học tập
- hỗ trợ cá nhân hóa việc học